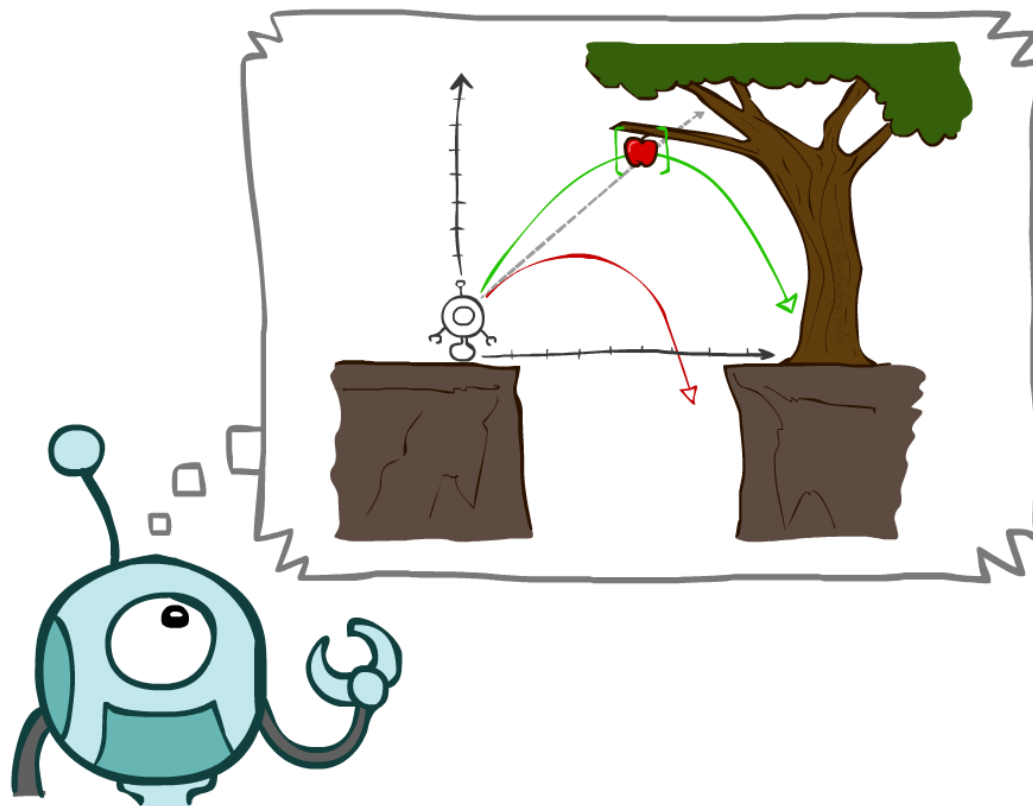




第二章：搜索

目录

- 搜索算法基础
- 无信息搜索（盲目搜索）
- 有信息搜索（启发式搜索）
- 局部搜索
- 约束满足问题



局部搜索

- 爬山法
- 模拟退火
- 局部束搜索
- 遗传算法
- 连续空间的局部搜索——梯度下降

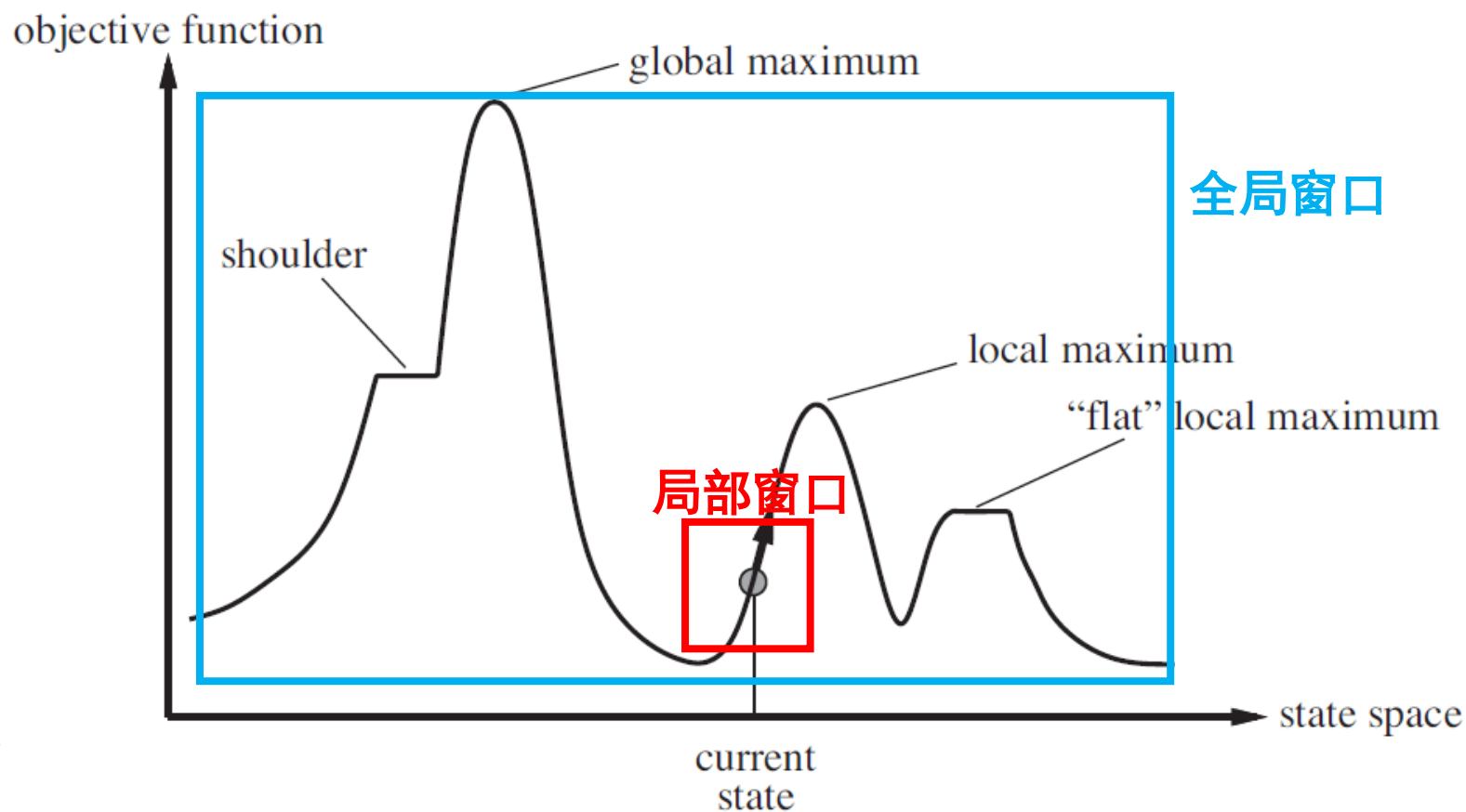
- 无信息/有信息是根据算法是否知道“离目标的远近”的线索来分的（有无指南针）
- 系统搜索/局部搜索是根据算法如何管理内存和探索路径来分的（怎么走路，记不记路）：
 - ✓ 系统搜索维护一个边缘，它记得走过的路，能回溯，目标是找到一条完整的路径
 - ✓ **局部搜索**通常只关注当前状态及其邻居，它不关心怎么走过来的，目标是找到一个最优的状态（山顶）
 - ✓ 局部搜索通常本身就是有信息的，因为如果没有一个评估函数来告诉你哪个邻居更好，局部搜索就根本没法找最好（爬山）

局部搜索1：爬山法

- 做法：不断地向**增加值最快**的方向移动，直到到达一个“峰值”，即没有邻居有更高的值
- 算法不维护搜索树，不会考虑当前状态不相邻的状态，类似在大雾中登山，只要找到向上的路就一直向上

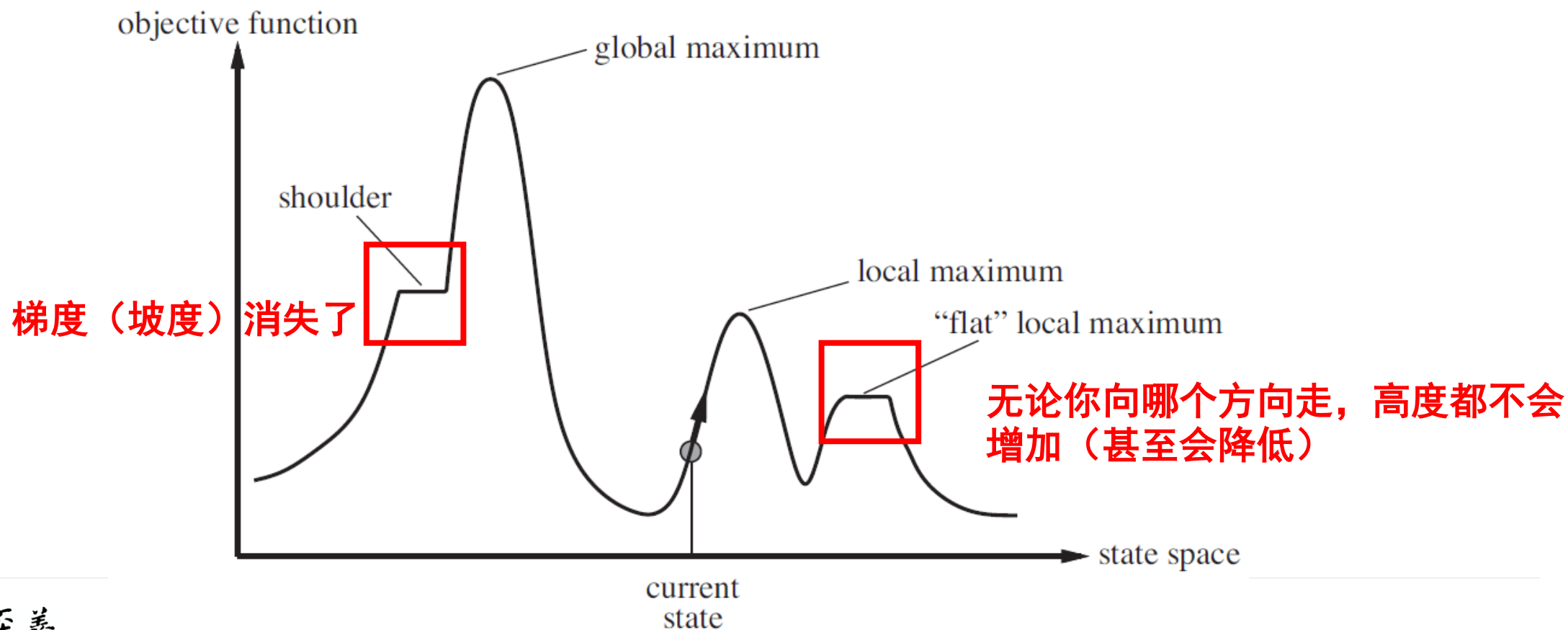
爬山法的问题

1. 局部最大值：局部极大值是一个峰值，它比相邻的每个状态都高，但比全局极大值低



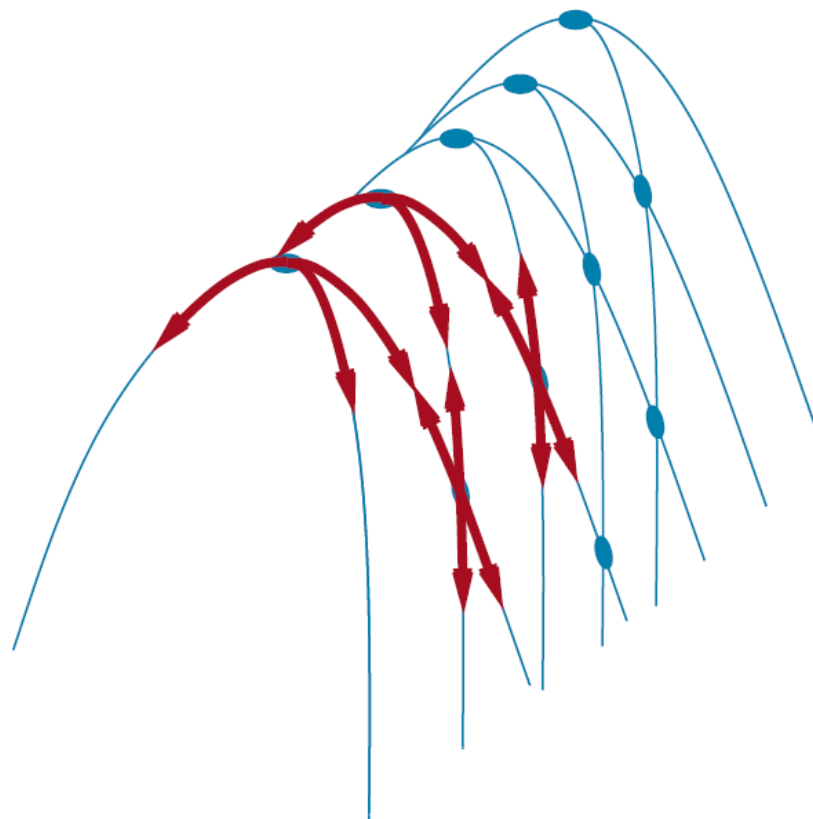
爬山法的问题

2. 高原：状态空间景观中一片极其平坦的区域。在这里，所有的邻居解的评估值都几乎一样，导致算法“迷失方向”



爬山法的问题

3. 岭：在状态空间景观中，岭是指一系列相对于东西方向是“高峰”，但相对于南北方向却有“坡度”的连续高点（或者反之）

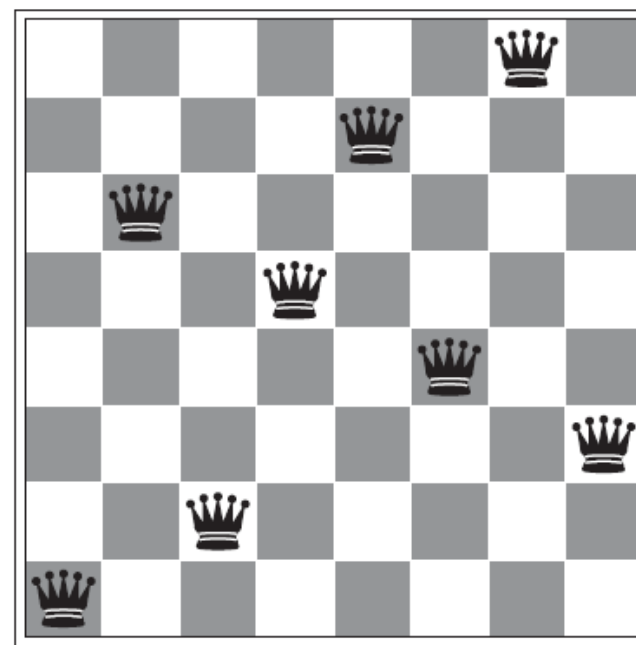


爬山法解8皇后问题

- 每个状态都在棋盘上放八个皇后，每列一个
- 后继函数返回移动一个皇后到和它同一列的另一个方格中的所有可能的状态
- 每个状态有 $8 \times 7 = 56$ 个后继
- 启发式函数 h 是可以彼此攻击的皇后对的数量
- $h = 0$ 是问题的解

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18

$h = 17$



$h = 1$

5 steps

local
maxima

心於至善

爬山法解8皇后问题



- 从一个随机生成的八皇后问题的状态开始，最陡上升的爬山法86%的情况下会被卡住，只有14%的问题实例能求解
- 这个算法速度很快，成功找到最优解的平均步数是4步，被卡住的平均步数是3步
- 对于 $8^8 \approx 17$ 万个状态的状态空间，这是不错的结果

- 随机爬山法：在上坡行动中随机选择一个，被选中的概率和上坡陡度有关
- 首选爬山法：在后继状态众多时，只要看到比当前好的就走
- 随机重启爬山法：从随机生成的初始状态开始爬山，如果卡住了，就随机换个地方重新爬，直到找到目标
- 爬山法虽然是不完备的（即不一定能找到解），但它代表了局部搜索的核心思想：用极小的空间开销，在复杂的概率景观中寻找最优的可能

局部搜索2：模拟退火

退火：为了增强金属的韧性和硬度，将金属加热到高温然后逐渐冷却，使材料达到低能量结晶态



局部搜索2：模拟退火

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to “temperature”

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

for $t = 1$ **to** ∞ **do**

$T \leftarrow \text{schedule}(t)$ T随着算法进行变化，一般逐渐减小

if $T = 0$ **then return** *current*

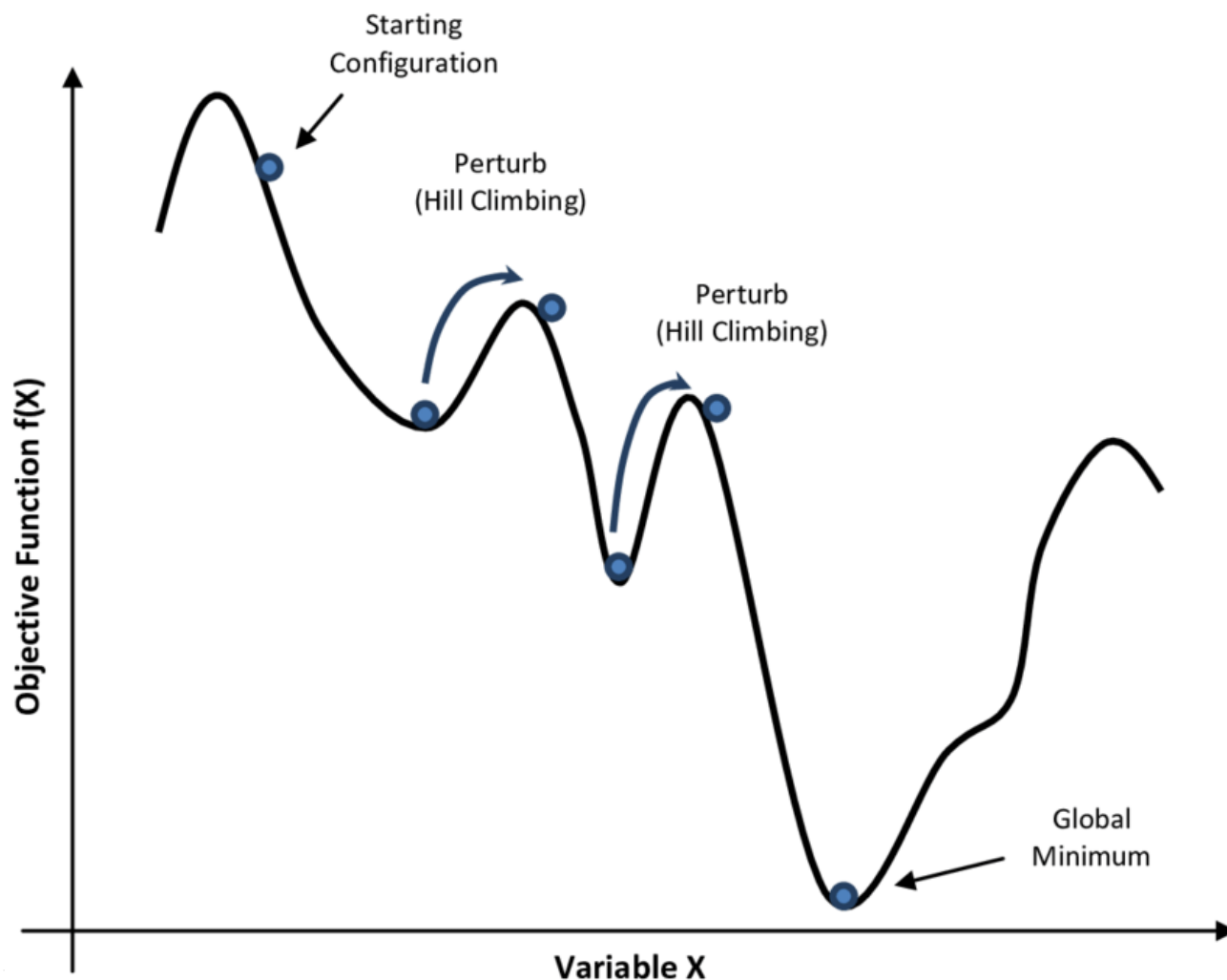
next $\leftarrow$ a randomly selected successor of *current* 随机选择一个后继

$\Delta E \leftarrow \text{next.VALUE} - \text{current.VALUE}$

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next* 若有效则移动

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$ 否则以小于1的概率接受

局部搜索2：模拟退火



- 如果只让乒乓球滚动，它会停留在局部极小点
- 如果晃动平面，可以使其弹出局部极小点
- 技巧是晃动足够大，使其能从局部极小点弹出来，但又不能太大，把它从全局极小点赶出来

局部搜索3：局部束搜索

- 目前我们看到的局部搜索方法只关注当前状态及其邻居，而只在内存中保留一个节点似乎是对内存限制问题的一种极端应对措施
- 局部束搜索：跟踪 k 个状态



局部搜索3：局部束搜索

随机选择 k 个状态

for 生成所有 k 个状态的所有后继

若有一个是目标则停止

否则从整个后继列表里选 k 个最优的后继

对比随机重启爬山法：

for 随机生成1个状态

for 生成1个后继

是目标则停止

否则 有路就继续爬

没有就跳出整个循环，重新开始

局部搜索3：局部束搜索



局部束搜索是并行k次随机重启搜索吗？

- 在局部束搜索中，有用的信息会在并行搜索的线程之间传递，而在随机重启搜索中，每次搜索都是独立进行的
- 局部束搜索会迅速放弃无果的搜索，并将资源转移到进展最大的地方
- 局部束搜索可能会因 k 个状态之间缺乏多样性而受到影响——它们可能会聚集在状态空间的一个小区域内，从而使搜索几乎只是比爬山算法慢 k 倍而已

有信息/启发式搜索：拥有辅助信息，一种关于“当前状态距离目标有多远”的知识

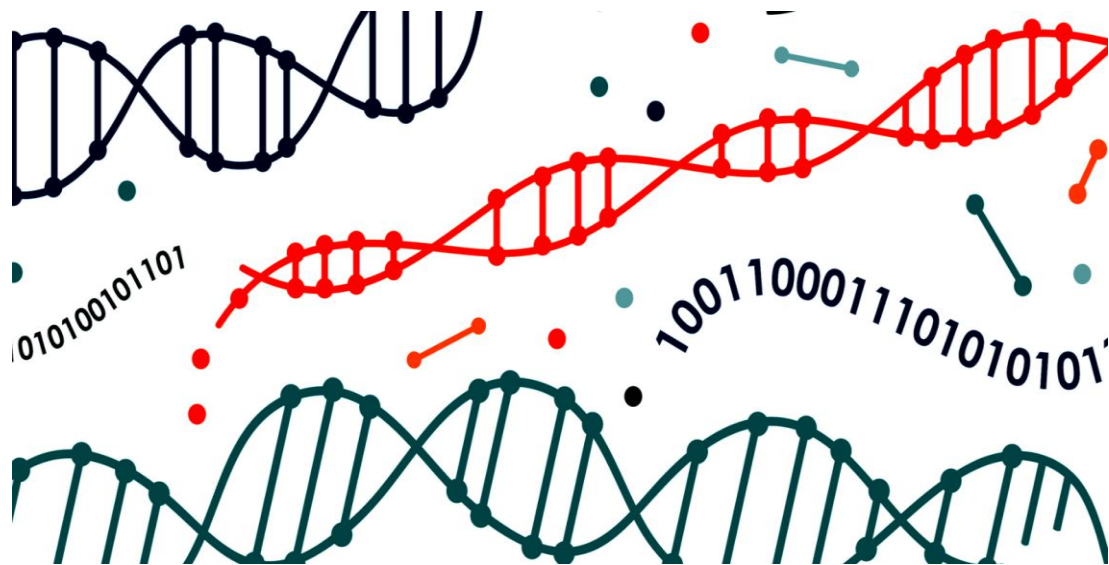
- 贪心最佳优先搜索：考虑未来 评价函数=启发函数
- A*搜索：考虑历史和未来
- 树搜索A*算法的最优性
- 图搜索A*算法的最优性
- 如何设计与评价启发函数

局部搜索：只关注当前状态及其邻居，它不关心怎么走过来的，目标是找到一个最优的状态

- 爬山法：很快，容易陷入局部最优->随机/首选/随机重启爬山法
- 模拟退火：当前看来不好的选择也有概率被选择
- 局部束搜索：每次保留 k 个状态

局部搜索4：遗传算法

- 遗传算法是局部束搜索的变体
- 后续状态是通过将两个父状态组合而成的，而非通过对单个状态进行修改来生成的

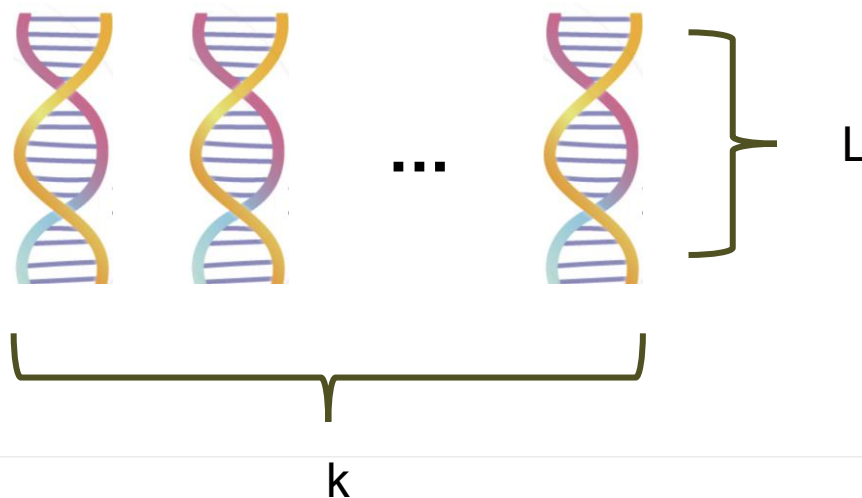


局部搜索4：遗传算法

- 种群：一次性生成 k 个随机状态，每个状态称为个体
- 适应度函数：评定状态的目标函数，对于好的状态，适应度函数应该返回较高的值
- 杂交：希望将两个优秀的基因片段拼接在一起
- 变异：改变其中一些元素，从而探索全新的未知区域

局部搜索4：遗传算法

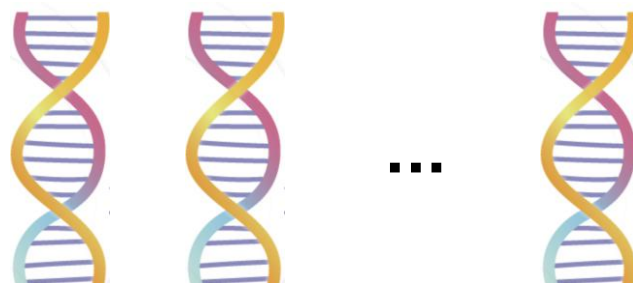
1. 随机产生 k 个状态，称为种群，对每个状态计算适应度函数
- ✓ 随机生成一组DNA，每个DNA上有 L 个基因，每个DNA都有自己对应的适应度，适应度最大的DNA为种群最优



局部搜索4：遗传算法

2. 选择子代，交叉重组

✓ 选取子一代



再生成 k 个DNA，这些DNA都是从初始种群中**按照适应度选取的**

✓ 交叉重组



随机地选择两个DNA的随机基因进行等位变换

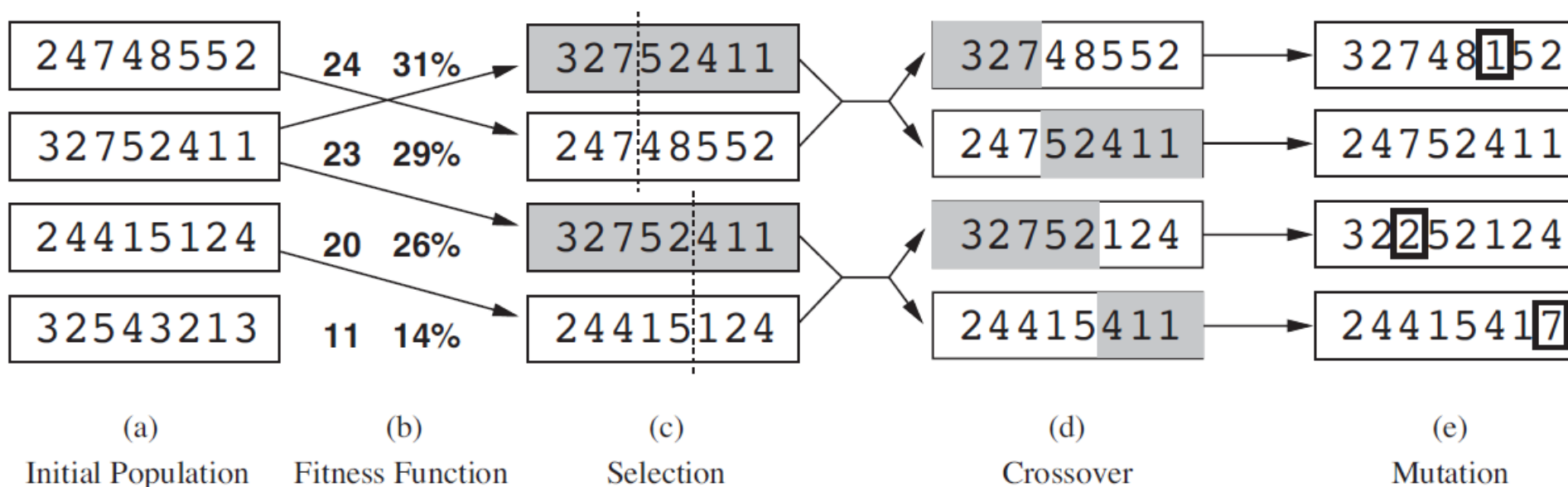
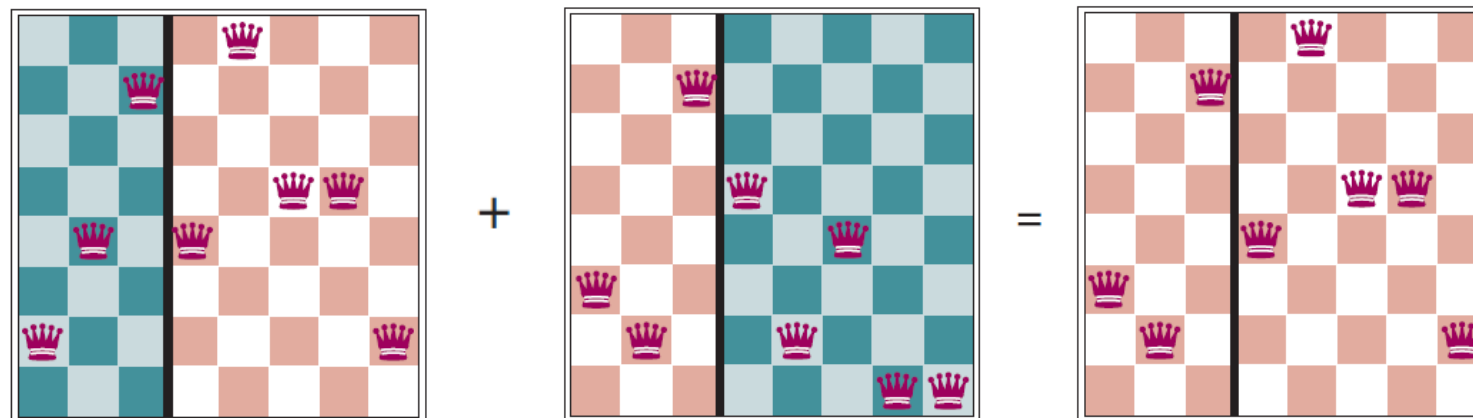
局部搜索4：遗传算法

3. 变异

- ✓ 随机地选择若干个DNA的若干个基因进行变异



局部搜索4：遗传算法



局部搜索4：遗传算法



- 遗传算法结合：上山趋势（适应度函数高的）、随机搜索（变异）、并行信息交换（杂交）
- 杂交的优势在于它能够将独立发展出来的、有用的字符区域（模式）结合起来。**模式**就是某些位没有确定的子串，如 246^{*****} ，能匹配这个模式的字符串，如24613578，称作该模式的一个实例
- 如果一个模式的实例的**平均适应度**值是超过均值的，则种群内该模式的实例数量会随时间增长，可以和其他有用的区域组合起来形成问题的解

Review



爬山法：贪心搜索，很快，会陷入局部最优

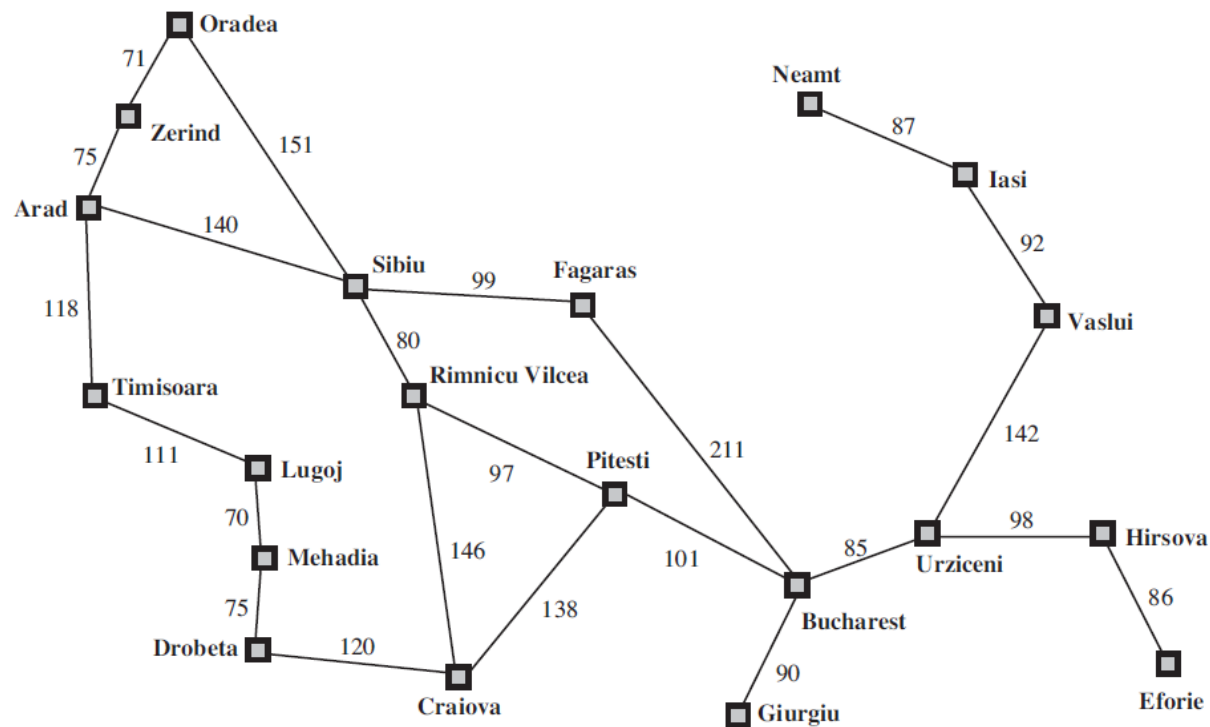
模拟退火：引入随机性！

局部束搜索：并行的信息交换

遗传算法：结合多种算法的优势

连续空间的局部搜索

如果我们的状态不是“格子”，而是连续的呢？



- 任务：在罗马尼亚的任意位置新建三个机场
- 目标：使地图上每个城市到其最近机场的距离平方和最小化

连续空间的局部搜索

策略1：离散化

- 对每个状态的邻域进行离散化处理
- 将一个机场向任意方向移动固定距离 $\pm \delta$
- 应用之前描述过的任何局部搜索算法

问题：效率低下，在高维空间会遭遇“维度灾难”

启发：能不能利用函数本身的“坡度”信息？

深度学习的优化方法 —— 梯度下降

机器学习的本质：找函数

Step 1: 假设
一个函数，
其中带有未
知参数



Step 2: 根据
数据定义衡
量函数好坏
程度的损失



Step 3: 求解
函数中的未
知参数

深度学习的优化方法 —— 梯度下降

1. 定义一个有未知参数的函数，将特征映射到目标

$$y = f(x)$$

函数根据先验知识来设定，最常用的是神经网络

$$y = \underset{\text{bias}}{b} + \underset{\text{weight}}{w}x$$

目标是求解函数中的未知参数

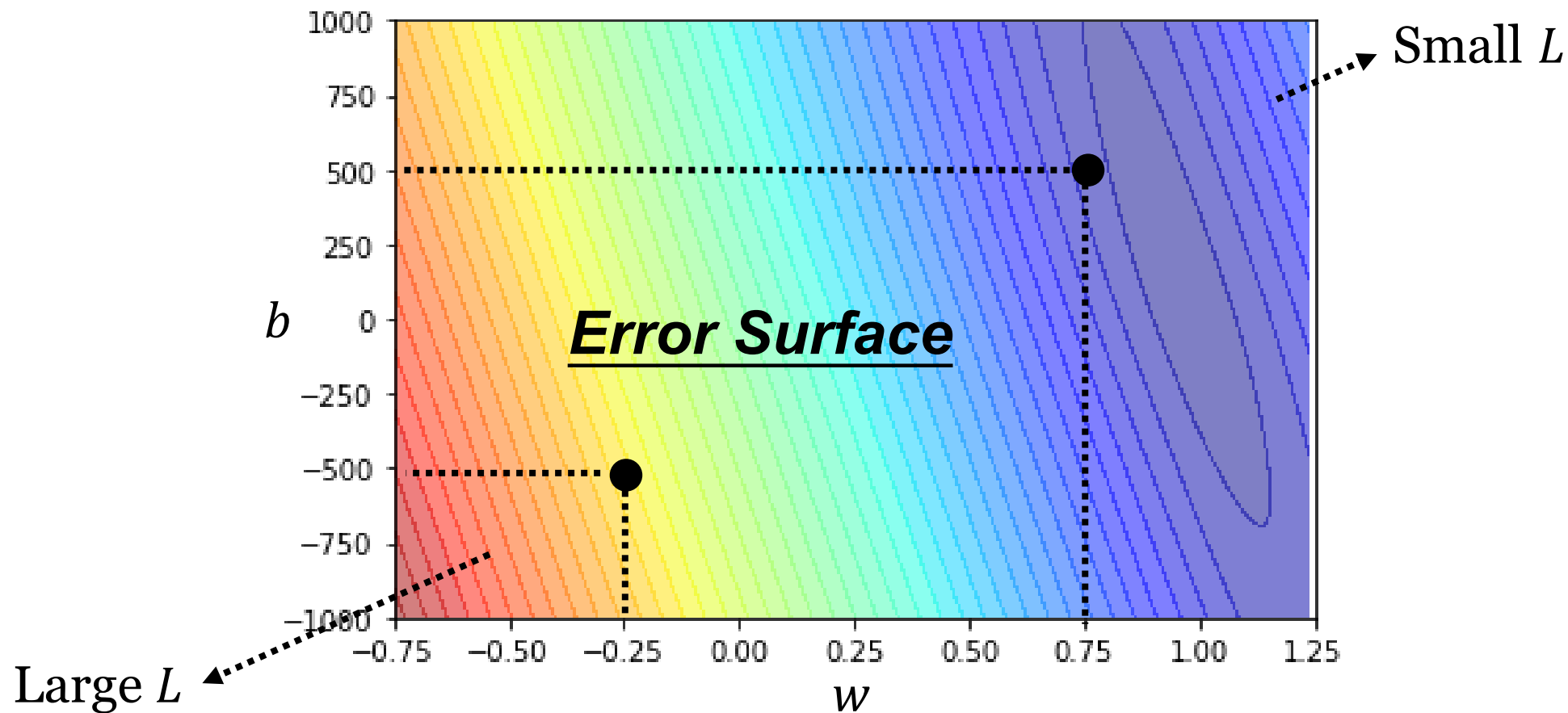
梯度下降

2. 根据训练数据，定义损失函数，用来衡量函数的好坏

Size in feet ² (x)	Price (\$) in 1000's (y)		$y = b + wx$ $b^0 = 0, w^0 = 0.25$	$L = \frac{1}{N} \sum_n e_n = 291.5$
2104	460	←.....→	$\hat{y} = 526$	$e_1 = y - \hat{y} = 66$
1416	232	←.....→	$\hat{y} = 354$	$e_2 = y - \hat{y} = 122$
1534	315	←.....→	$\hat{y} = 383.5$	$e_3 = y - \hat{y} = 68.5$
852	178	←.....→	$\hat{y} = 213$	$e_4 = y - \hat{y} = 35$
...	...			
$b^0 = -20, w^0 = 0.23 \quad L = 115.68$				
			$\hat{y} = 465.92$	$e_1 = y - \hat{y} = 5.92$
			$\hat{y} = 305.68$	$e_2 = y - \hat{y} = 73.68$
			$\hat{y} = 332.82$	$e_3 = y - \hat{y} = 17.82$
			$\hat{y} = 175.96$	$e_4 = y - \hat{y} = 17.96$

梯度下降

2. 根据训练数据，定义损失函数，用来衡量函数的好坏



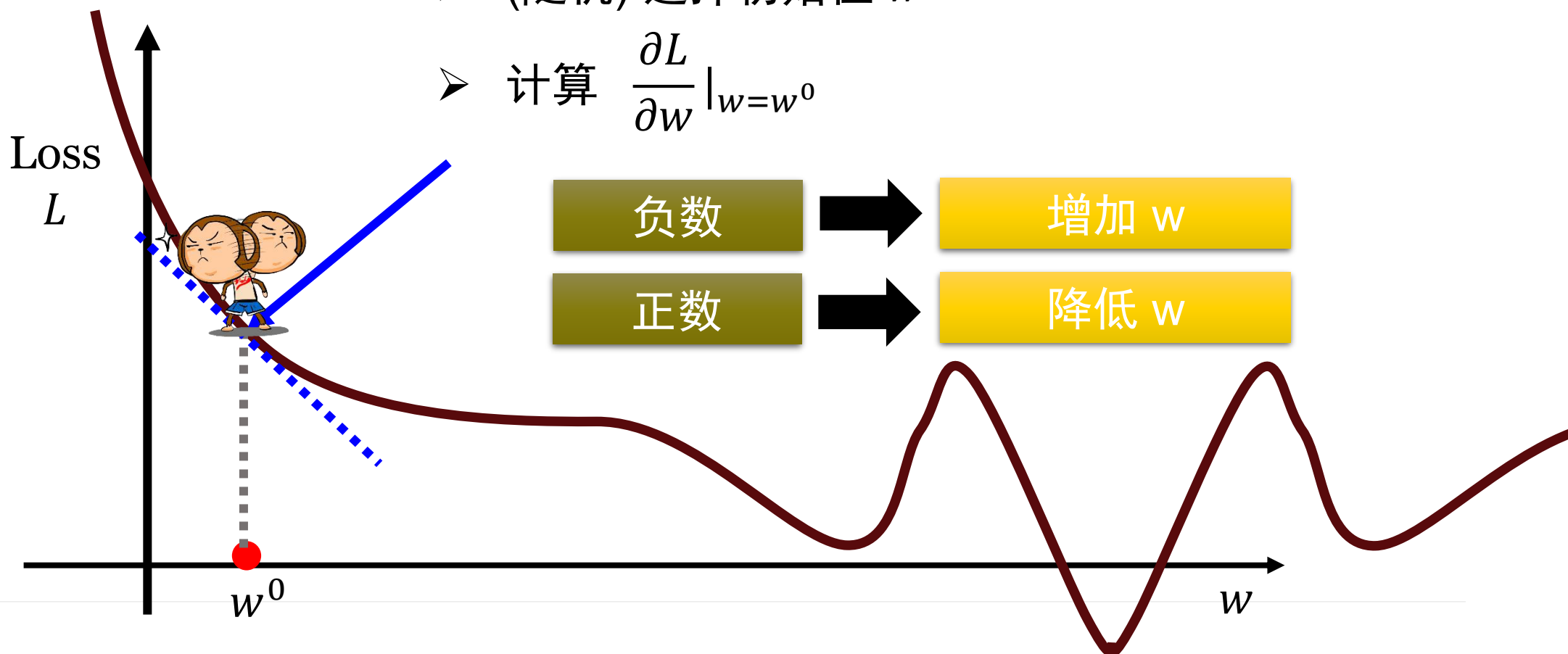
梯度下降

3. 优化

$$w^* = \arg \min_w L$$

➤ (随机) 选择初始值 w^0

➤ 计算 $\frac{\partial L}{\partial w} \big|_{w=w^0}$



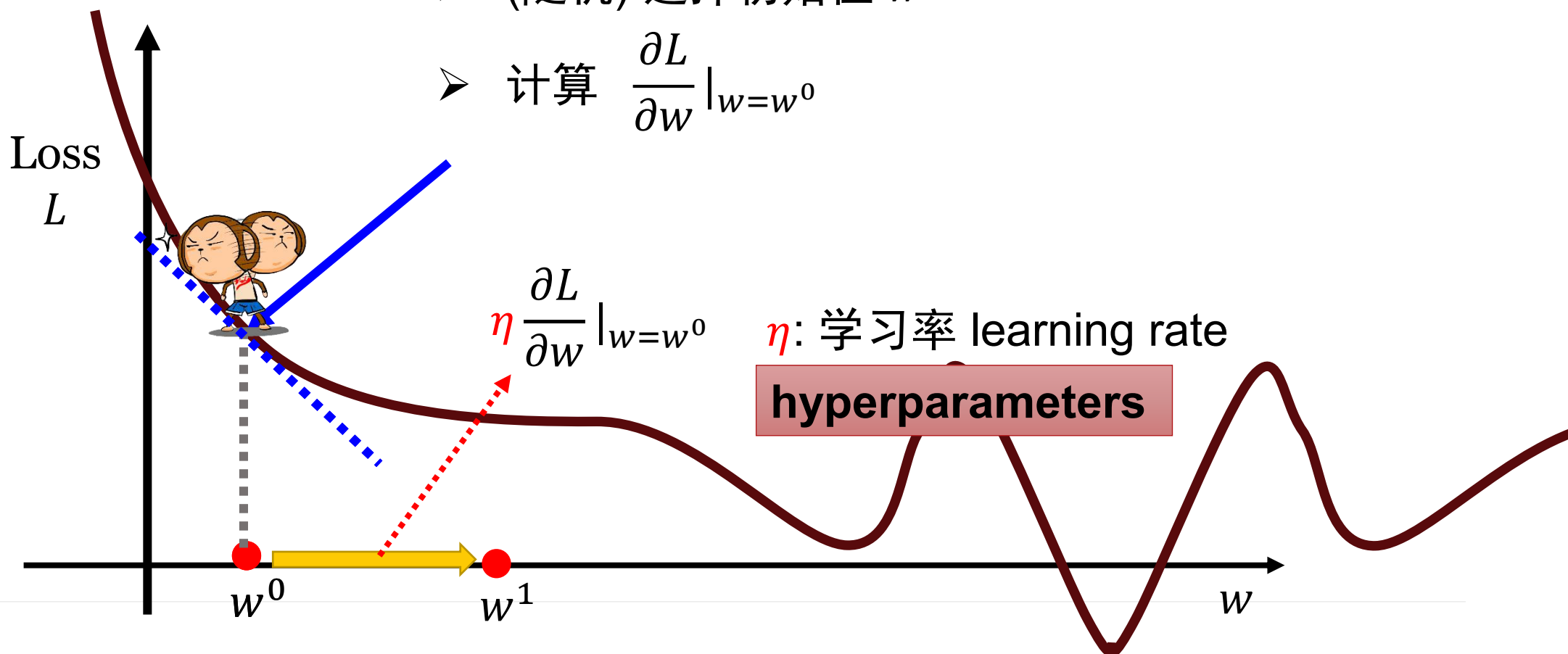
梯度下降

3. 优化

$$w^* = \arg \min_w L$$

➤ (随机) 选择初始值 w^0

➤ 计算 $\frac{\partial L}{\partial w} \big|_{w=w^0}$



梯度下降

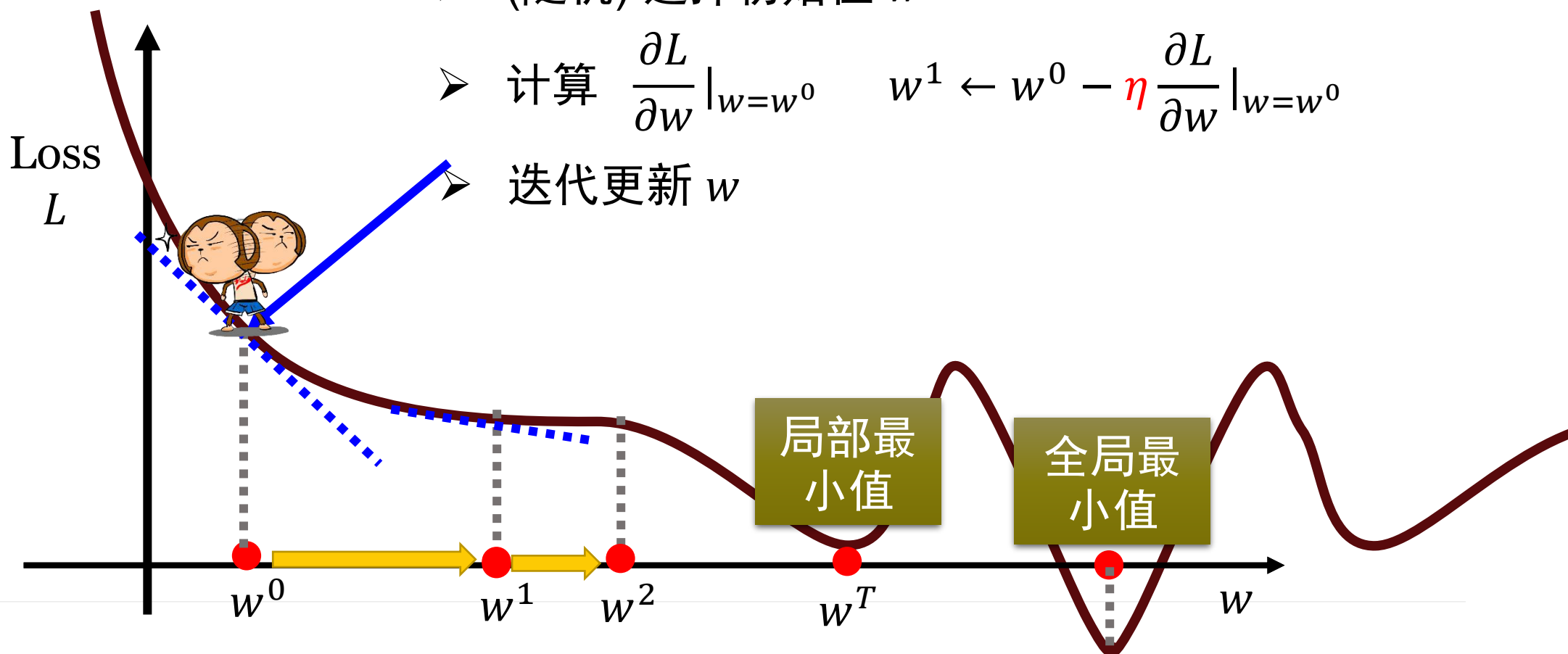
3. 优化

$$w^* = \arg \min_w L$$

➤ (随机) 选择初始值 w^0

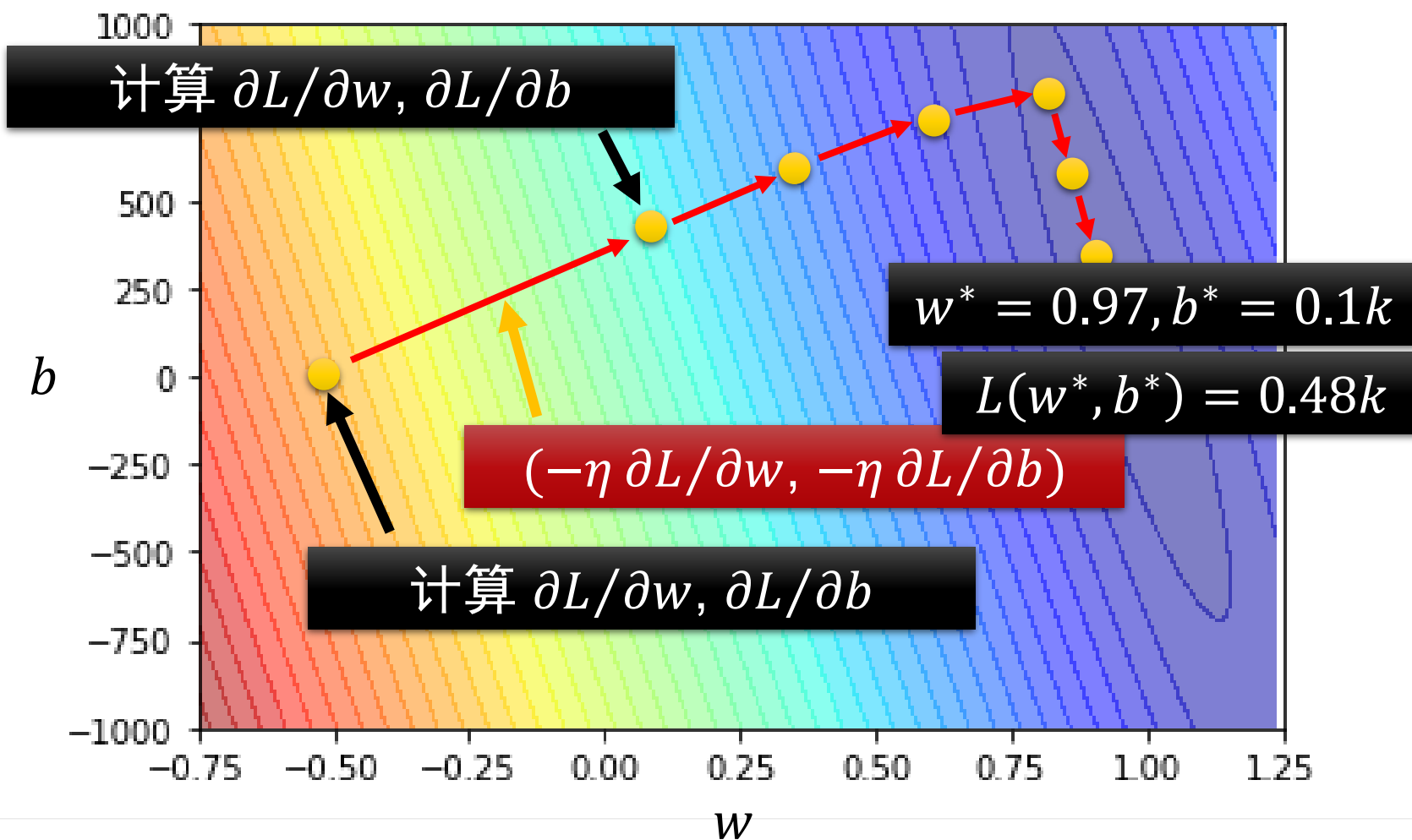
➤ 计算 $\frac{\partial L}{\partial w} \big|_{w=w^0}$ $w^1 \leftarrow w^0 - \eta \frac{\partial L}{\partial w} \big|_{w=w^0}$

➤ 迭代更新 w



梯度下降

3. 优化



梯度下降

形式化地说，在步骤3中，我们需要解决以下优化问题：

$$\theta^* = \arg \min_{\theta} L(\theta) \quad L: \text{loss function} \quad \theta: \text{parameters}$$

假设 θ 有两个值 $\{\theta_1 \theta_2\}$

从一个随机的初始值 $\theta^0 = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix}$ 开始

迭代

$$\nabla L(\theta) = \begin{bmatrix} \partial L(\theta_1)/\partial \theta_1 \\ \partial L(\theta_2)/\partial \theta_2 \end{bmatrix}$$

$$\begin{bmatrix} \theta_1^1 \\ \theta_2^1 \end{bmatrix} = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix} - \eta \begin{bmatrix} \partial L(\theta_1^0)/\partial \theta_1 \\ \partial L(\theta_2^0)/\partial \theta_2 \end{bmatrix}$$

$$\begin{bmatrix} \theta_1^2 \\ \theta_2^2 \end{bmatrix} = \begin{bmatrix} \theta_1^1 \\ \theta_2^1 \end{bmatrix} - \eta \begin{bmatrix} \partial L(\theta_1^1)/\partial \theta_1 \\ \partial L(\theta_2^1)/\partial \theta_2 \end{bmatrix}$$

随机梯度下降

梯度下降

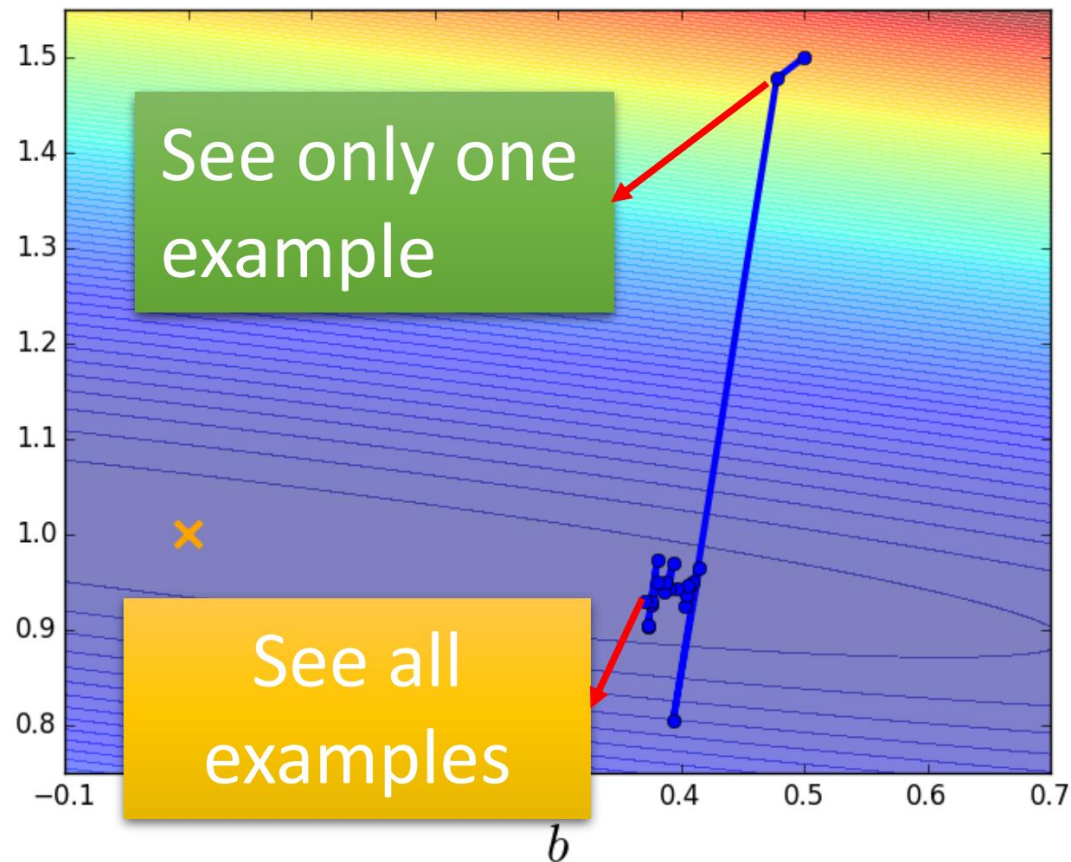
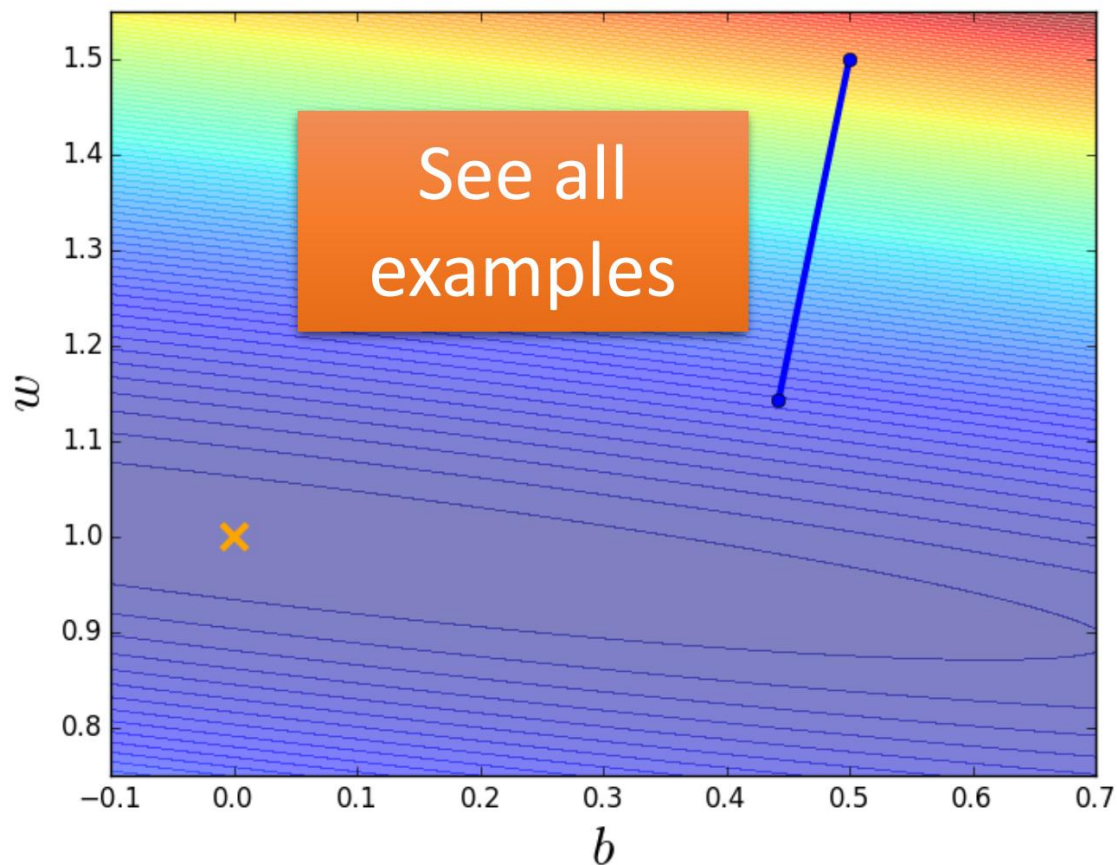
$$L = \sum_n \left(\hat{y}^n - \left(b + \sum w_i x_i^n \right) \right)^2$$
$$\theta^i = \theta^{i-1} - \eta \nabla L \left(\theta^{i-1} \right)$$

随机梯度下降

$$L = \left(\hat{y}^n - \left(b + \sum w_i x_i^n \right) \right)^2$$
$$\theta^i = \theta^{i-1} - \eta \nabla L^n \left(\theta^{i-1} \right)$$

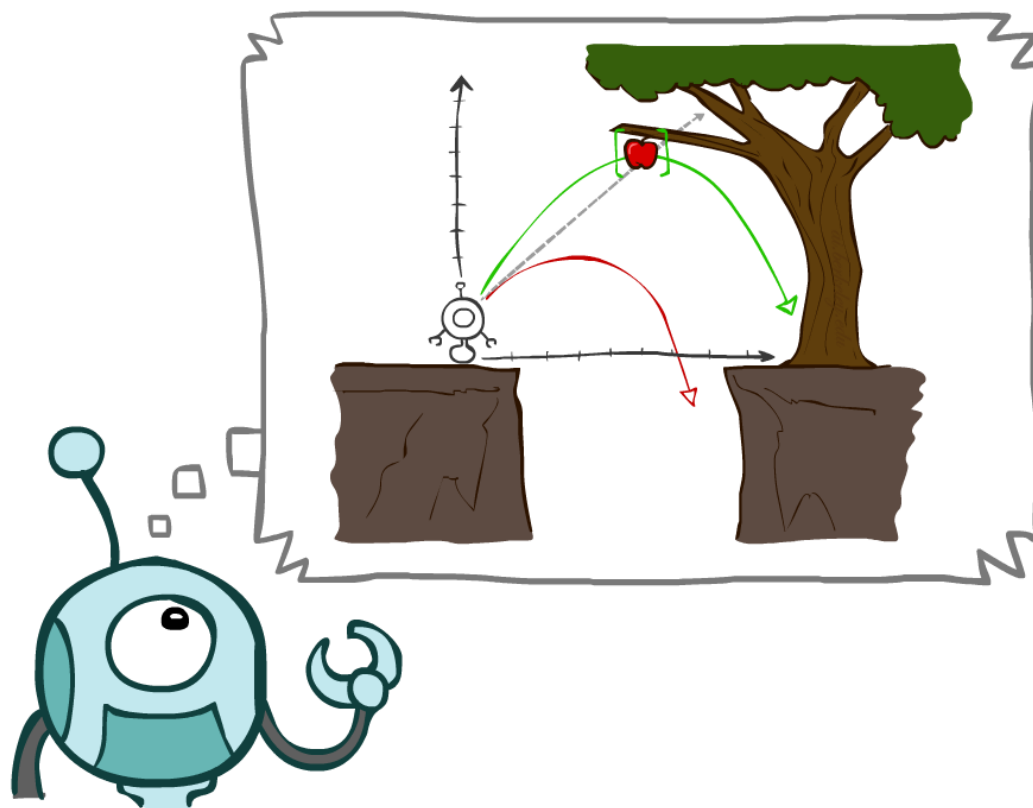
更快！

随机梯度下降



目录

- 搜索算法基础
- 无信息搜索（盲目搜索）
- 有信息搜索（启发式搜索）
- 局部搜索
- 约束满足问题



约束满足问题



- 问题定义
- 约束传播
- 回溯法求解
- 局部搜索求解
- 利用问题的结构信息求解

约束满足问题

在现实世界中，很多问题（如排班、配置、解码）的重点不在于“如何到达目标”，而在于“目标必须长成什么样”

- 标准搜索问题：

- ✓ 状态是一个黑盒，算法只能通过目标测试函数来判断“是”或“否”
- ✓ 每个状态是原子的、不可分割的

- 约束满足问题：

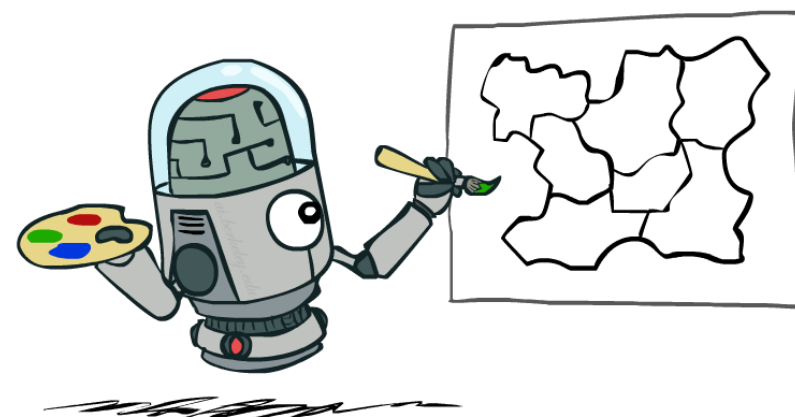
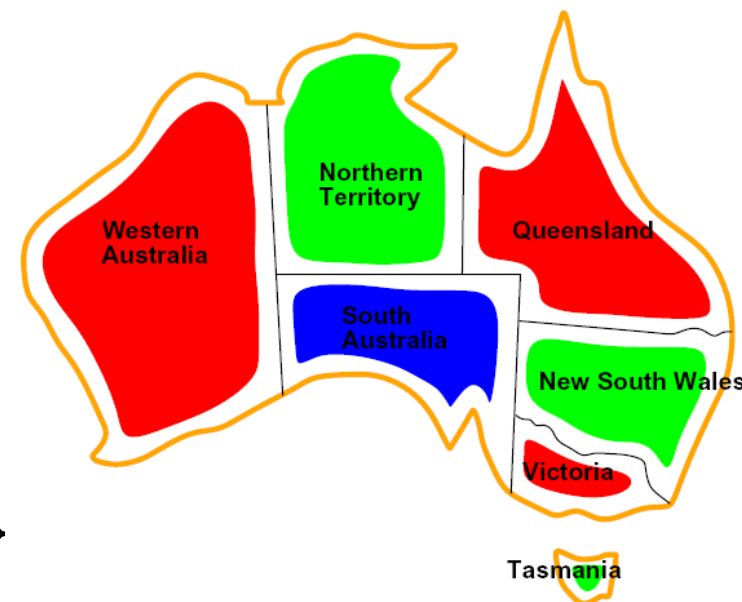
- ✓ 将状态分解为一组变量，每个变量在特定的值域中取值
- ✓ 不再仅仅依靠一个黑盒评估函数，而是定义一系列显式的约束

约束满足问题

- 三要素
 - ✓ 变量 $\{X_1, \dots, X_n\}$
 - ✓ 每个变量的值域 $\{D_1, \dots, D_m\}$
 - ✓ 约束 $C = \langle \text{范围}, \text{关系} \rangle$ 指定允许的值的组合
- 一个状态就是特征向量中变量的一个赋值组合，也就是关于一个事件或对象的描述
- 当特征向量中只有部分变量有赋值，称为部分赋值

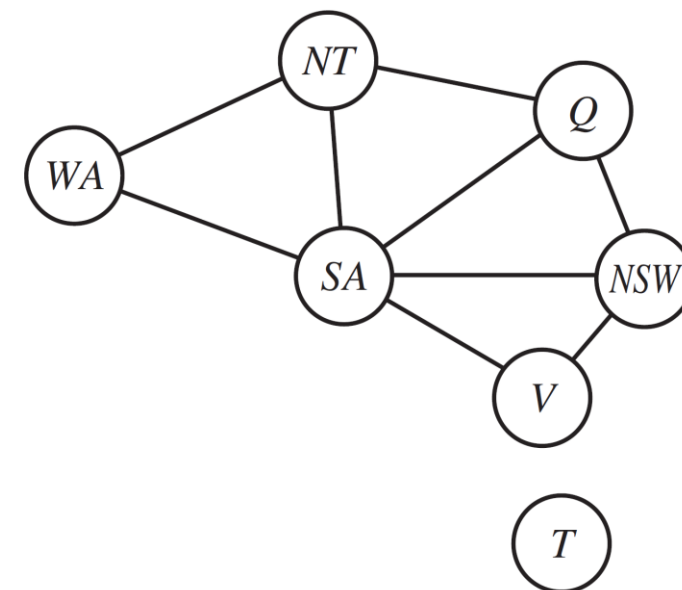
约束满足问题

- 变量: WA, NT, Q, NSW, V, SA, T
- 值域: $D = \{\text{red, green, blue}\}$
- 约束: 邻接的州不能涂相同的颜色
 - ✓ 隐式表示: $SA \neq WA$
 - ✓ 显式表示: $(SA, WA) = \{(\text{red, green}), (\text{red, blue}), \dots\}$
- 解必须满足约束, 如
$$\{WA=\text{red}, NT=\text{green}, Q=\text{red}, NSW=\text{green}, V=\text{red}, SA=\text{blue}, T=\text{green}\}$$



建模约束满足问题

- 二元约束满足问题：每个约束最多涉及两个变量
- 如果引入足够多的辅助变量，任何高阶的、有限值域的约束都可以简化为一个二元约束满足问题
- 二元约束图：点代表变量，边代表约束
- 约束图定义问题的**结构信息**，结构信息极大地加速求解过程，这是约束满足问题求解的核心

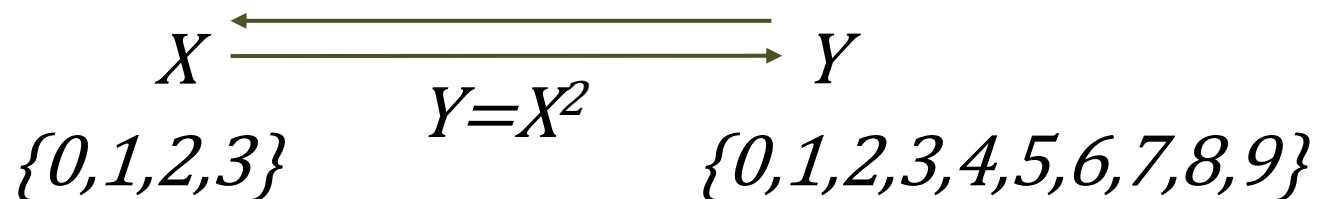


搜索算法中的关键操作：约束传播

- **约束传播**：利用约束条件来减少一个变量的合法取值数量，进而也减少其他变量的合法取值数量
- 其核心思想是局部一致性：在约束图的一部分**利用约束，删掉那些注定错误的选项**的过程中，影响会扩散至整个图
- 目标是在搜索中尽早发现矛盾，减小搜索空间

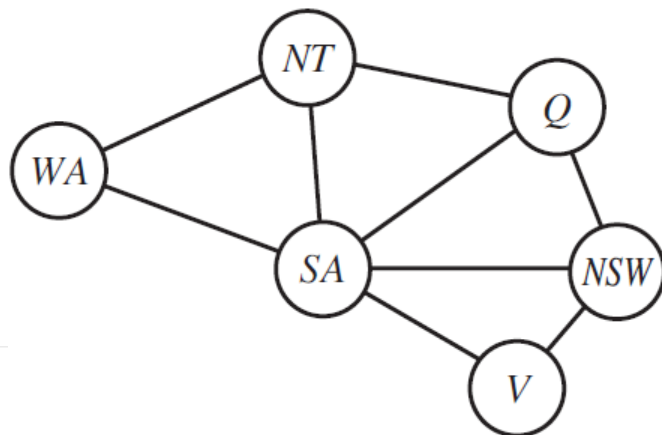
- 点一致：变量域内的所有值都满足该变量的一元约束
- 通过执行节点一致性操作，能够消除约束图中的所有一元约束
- 比如SA南澳大利亚人不喜欢绿色： $SA = \{\text{red}, \text{blue}, \text{green}\}$

- 弧一致：值域中的每个值都满足该变量的二元约束
- X_i 对 X_j 弧一致： D_i 中的每个值， D_j 中都有对应的取值，使得两个变量间的二元约束被满足
- 即，对于弧尾（弧的出发点）变量的任何一个取值，我们总能够在弧首（弧指向的点）变量的值域中找到一个赋值，使得这一对变量赋值能够满足二者之间所有的约束条件



路径一致

- 如果说弧一致是检查“两个变量”之间的矛盾，那么路径一致就是通过“第三个变量”作为中介，来检查“两个变量”之间是否真的一致
- 弧一致只管X和Y能不能凑一对，路径一致管得更宽：对于X和Y之间每一组“看起来合法”的取值（WA=红 SA=蓝），是否存在一个Z（NT）的取值，X和Z也是合法的，且Y和Z也是合法的？
- 对这两种赋值NT不存在有效选择，所以我们消除这种赋值



k一致

- k一致：如果对于任何k-1个变量的任何一致赋值，第k个变量总能被赋予一个和前k-1个变量一致的值，那么这个问题就是k一致的
- 强k一致： k一致且(k-1)且(k-2)一致且...1一致

在实际问题中，全局约束现象较为常见，并且可以通过专门的算法来检查

- **Alldiff** 约束：所涉及的所有变量必须具有不同的值

检查方法：如果该约束涉及 m 个变量，并且它们总共具有 n 种可能的不同值，且 $m > n$ ，那么该约束就无法得到满足

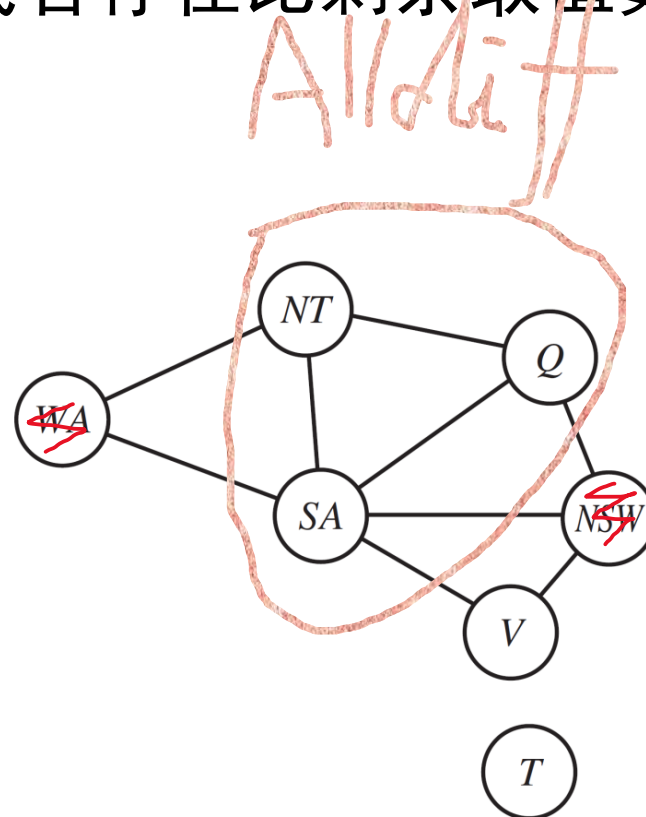
全局约束

Alldiff 约束传播的经典实现:

for 存在单值变量 (域中只有一个值的变量)

删除任意一个单值变量, 并且从其余变量的域中删除该变量的值

if 在任一点上产生了空集, 或者存在比剩余取值数更多的变量
检测到了不一致性



- **Atmost** 约束：资源限制

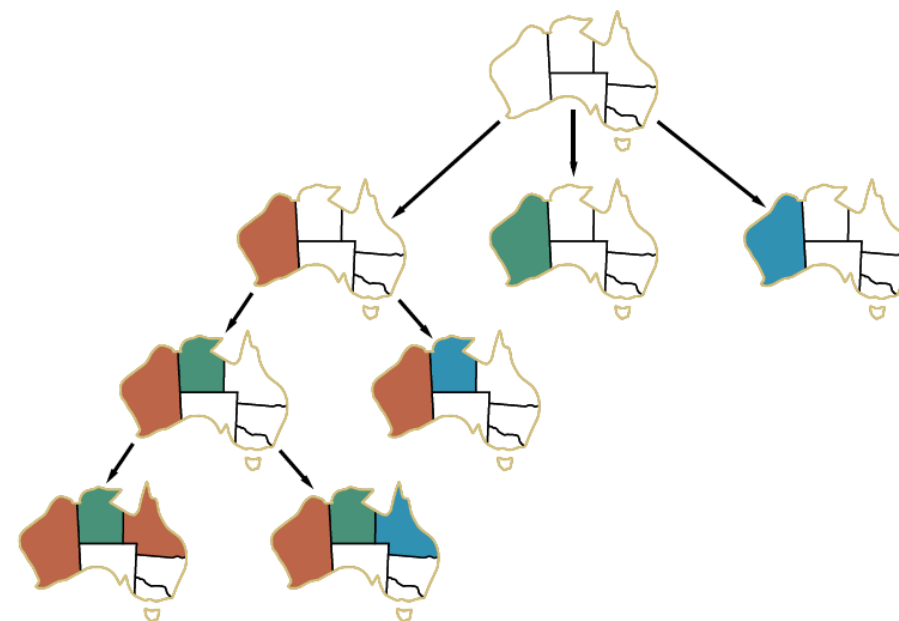
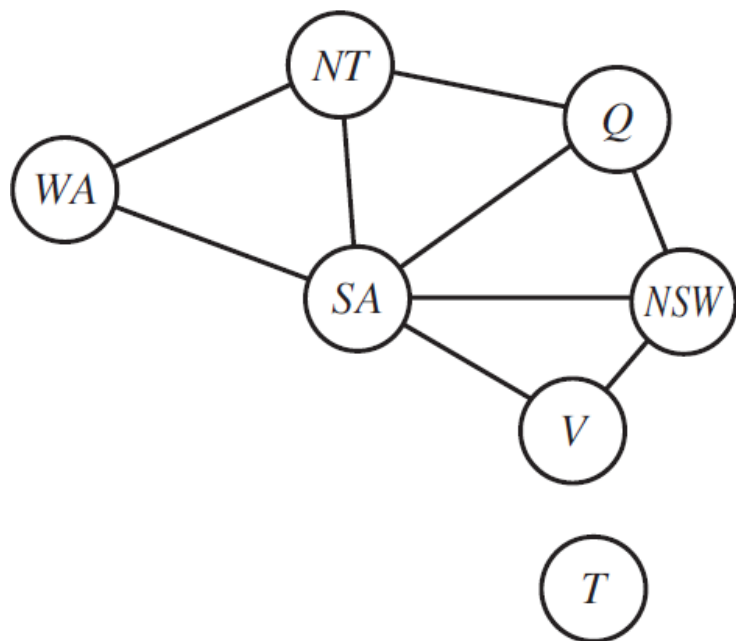
$\text{Atmost}(10;P1;P2;P3;P4)$ ：将10元分配给四个人 P1, P2, P3, P4

检查方法：当前值域的最小值之和

如果每个变量的值域都是 $\{3;4;5;6\}$ ， $3+3+3+3>10$ 无解

Atmost 约束传播的经典实现：检验当前值域中的最大值加上所有其他变量值域的最小值，消除不一致的取值。如如果每个变量的值域都是 $\{2;3;4;5;6\}$ ，那么从每个变量值域中删除 5 & 6 ($5+2+2+2>10$)

约束满足问题的搜索算法



- 广度优先搜索会做什么？
- 深度优先搜索会做什么？

在深度优先搜索的基础上进行改进

1. 变量排序，即按照**固定顺序**依次为变量赋值，每轮一个
2. 冲突停止，即变量赋值的过程中，**同时检查**约束条件，只考虑和现有赋值不存在冲突的取值

回溯搜索=深度优先搜索+变量排序+冲突停止

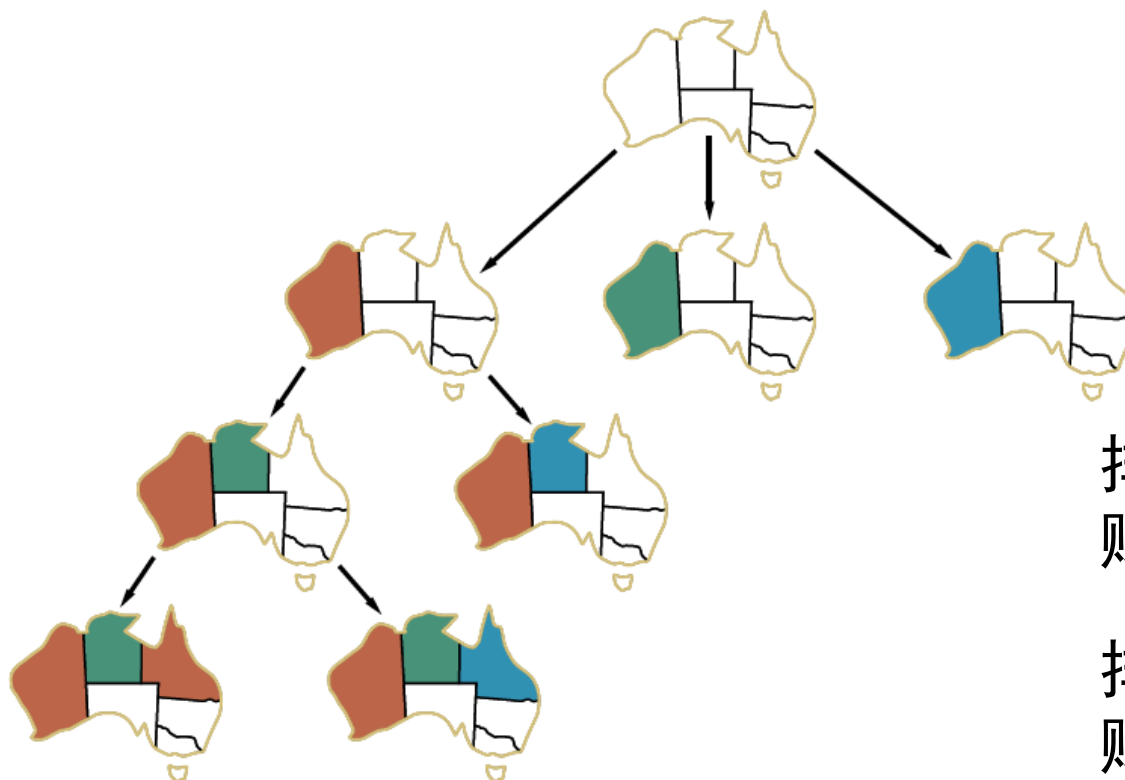
回溯搜索

空赋值

一轮赋值变量:WA

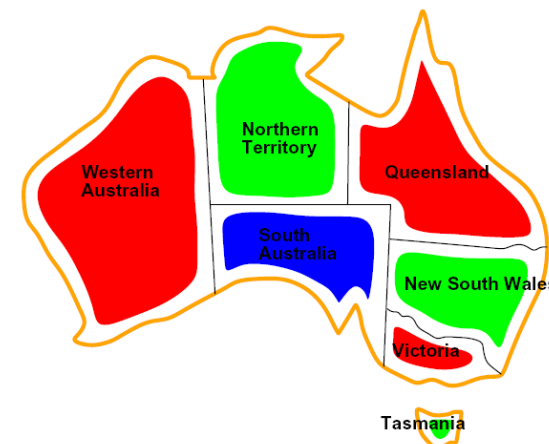
二轮赋值变量:NT

三轮赋值变量:Q



排除了NT与WA同色的
赋值组合

排除了Q与NT同色的
赋值组合



回溯搜索

function BACKTRACKING-SEARCH(*csp*) **returns** a solution or *failure*
return BACKTRACK(*csp*, {})

function BACKTRACK(*csp*, *assignment*) **returns** a solution or *failure*

if *assignment* is complete **then return** *assignment*

var $\leftarrow$ SELECT-UNASSIGNED-VARIABLE(*csp*, *assignment*) 取一个变量

for each *value* **in** ORDER-DOMAIN-VALUES(*csp*, *var*, *assignment*) **do** 遍历值域

if *value* is consistent with *assignment* **then** 检查当前这个取值是否满足与已赋值变量之间的约束，如果初步合法，先把它填进当前的解
add {*var* = *value*} to *assignment*

inferences $\leftarrow$ INFERENCE(*csp*, *var*, *assignment*) 推断：约束传播

if *inferences* $\neq$ *failure* **then** 如果推导发现后面还有戏（没有产生空值域）

add *inferences* to *csp* 将推导出的结果（比如值域的缩减）应用到当前的问题状态中

result $\leftarrow$ BACKTRACK(*csp*, *assignment*) 带着刚才的决定和推导出的新状态，递归进入下一层

if *result* $\neq$ *failure* **then return** *result* （去填下一个变量）

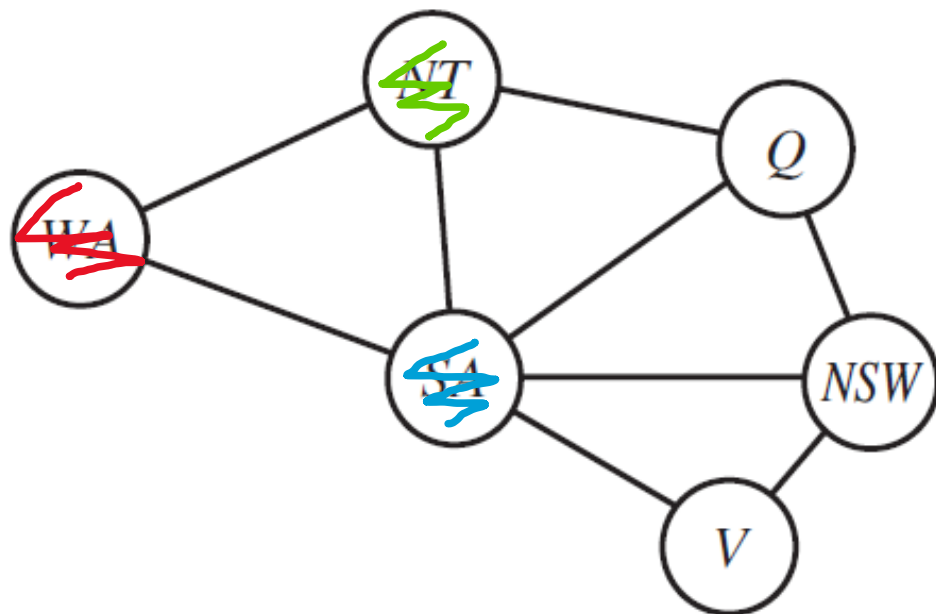
remove *inferences* from *csp* 回溯开始！如果这条路不通，先把刚才删掉的那些候选值给加回来

remove {*var* = *value*} from *assignment* 把刚才填错的那个值也“擦掉”

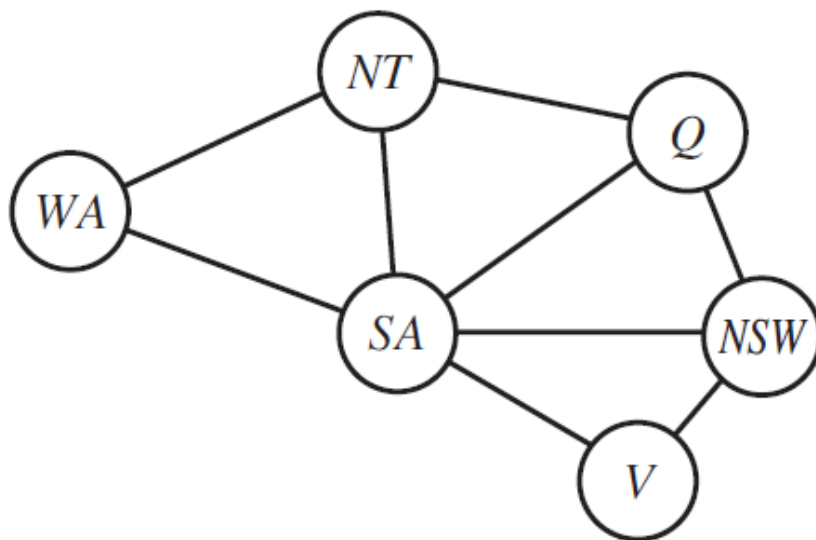
return *failure*

如果深层递归成功
找到了解，一路返
回成功

1. 最小剩余值启发式：选择具有最少合法值的变量
 - 也称为最失败优先启发，因为它选择了最可能很快导致失败的变量，从而快速对搜索树剪枝
 - SA or Q?

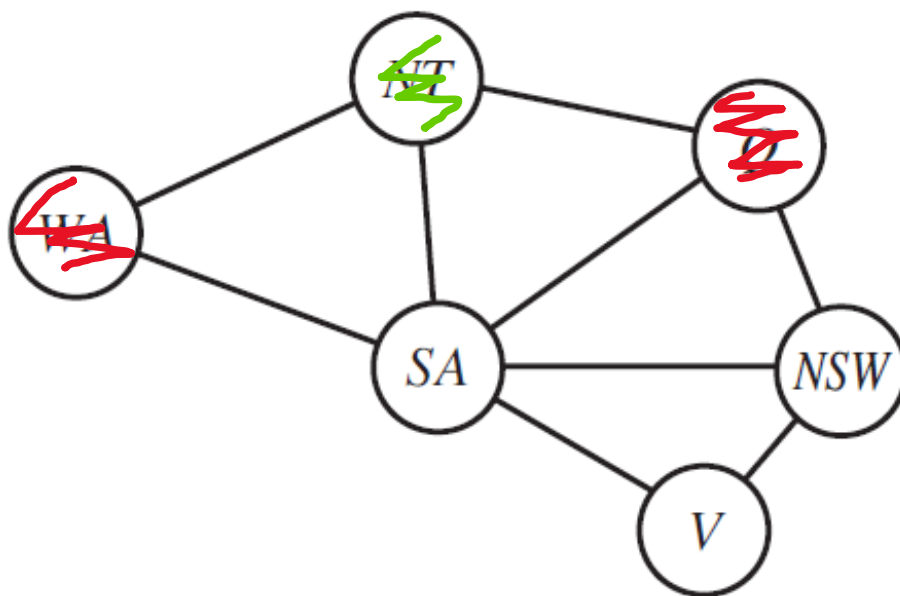


2. 度启发式：选择与其他未分配变量的**约束数量最多**的变量
- 最小剩余值启发式在澳大利亚着色问题中对选择第一个染色区域没有帮助，因为初始的时候每个区域都有三种合法的颜色
 - 度启发式可以打破僵局



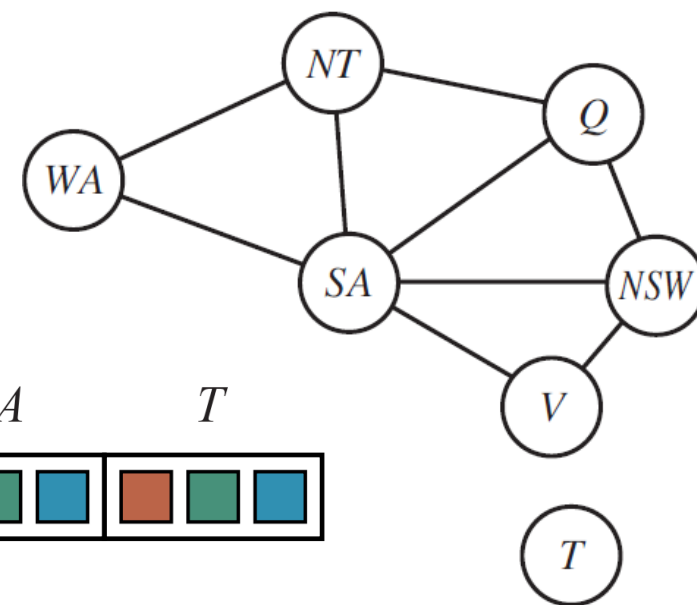
- 最少约束值启发式：给邻居变量留下更多选择

WA=red and NT=green and next choice is for Q. What is Q' color?



推断：约束传播

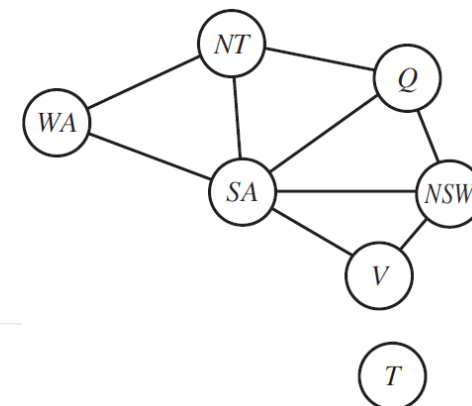
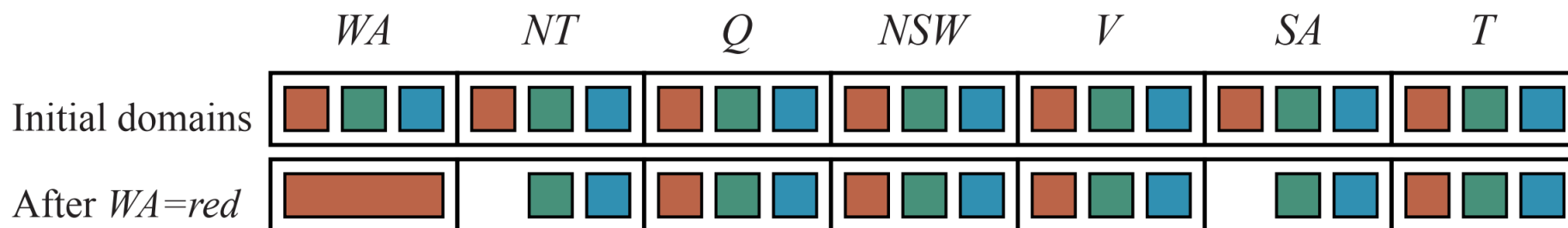
1. 前向检验：每轮变量赋值后，都调整**和这个变量相连的变量**的值域，将其中与该变量赋值相冲突的值都去掉



	WA			NT			Q			NSW			V			SA			T		
Initial domains																					
After $WA=red$																					
After $Q=green$																					
After $V=blue$																					

推断：约束传播

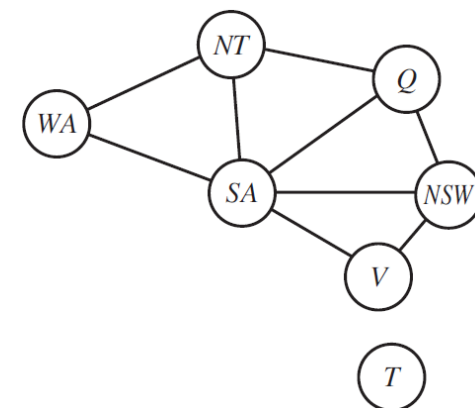
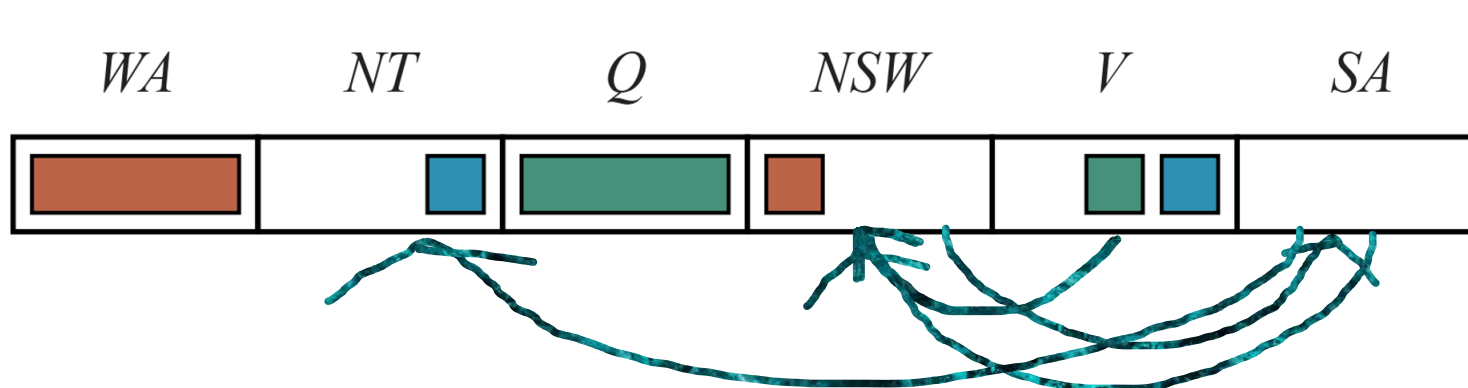
- 回顾弧一致：对于弧尾（出发点）变量的任何一个取值，我们总能够在弧首（指向点）变量的可行值域中找到一个赋值，使得这一对变量赋值能够满足二者之间所有的约束条件
- 前向检查用弧一致的方式表达：对新赋值的变量，做其它变量对（指向）当前变量的弧一致检查，即去掉弧尾变量中无法与赋值变量进行匹配的变量值



推断：约束传播

2. 维护弧一致MAC：当一个变量的值域发生变化后，**检查所有指向它的弧**

Q=green 并执行前向传播后：



- 如果x丢失一个值，那么x的所有邻居都需要被重新检查
- MAC可以比前向检验更早得发现错误
- MAC不仅可以用作搜索过程之前的预处理，也可以在搜索过程中每次赋值后用作一个传播约束步骤

AC-3

function AC-3(csp) **returns** false if an inconsistency is found and true otherwise

inputs: csp , a binary CSP with components (X, D, C)

local variables: $queue$, a queue of arcs, initially all the arcs in csp

while $queue$ is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(queue)$

if REVISE(csp, X_i, X_j) **then**

if size of $D_i = 0$ **then return** false

for each X_k **in** $X_i.\text{NEIGHBORS} - \{X_j\}$ **do**

add (X_k, X_i) to $queue$

Xi的域被修改，将指向Xi的弧加入到队列里

return true

function REVISE(csp, X_i, X_j) **returns** true iff we revise the domain of X_i

$revised \leftarrow false$

当且仅当我们修改Xi的域返回true

for each x **in** D_i **do**

if no value y in D_j allows (x, y) to satisfy the constraint between X_i and X_j **then**

delete x from D_i **将当前Xi的取值x从Di中删除**

如果Dj中不存在满足约束的值

$revised \leftarrow true$

return $revised$

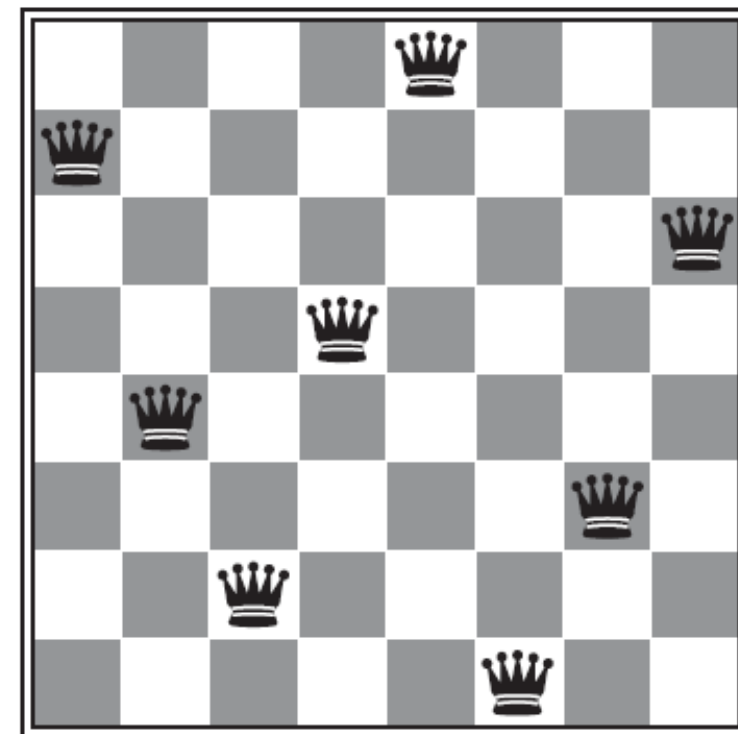
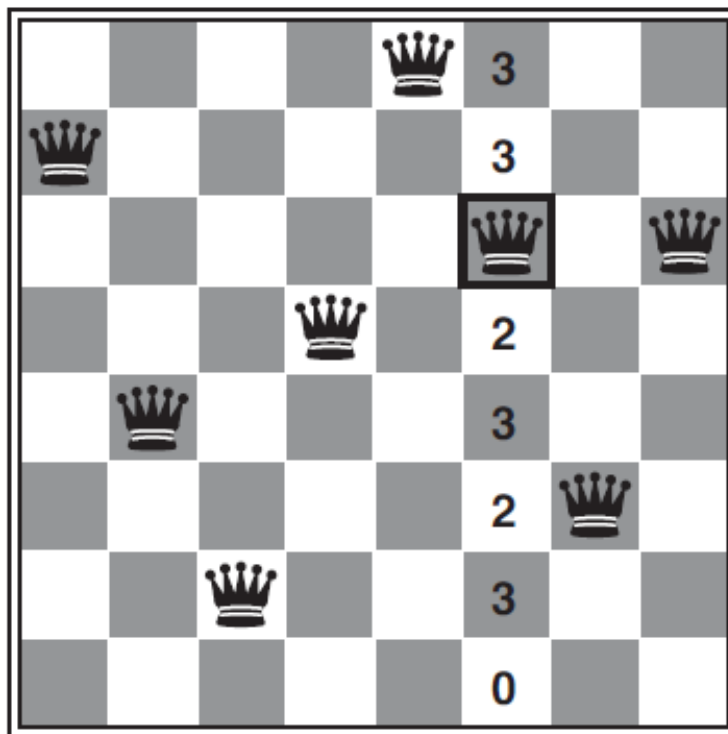
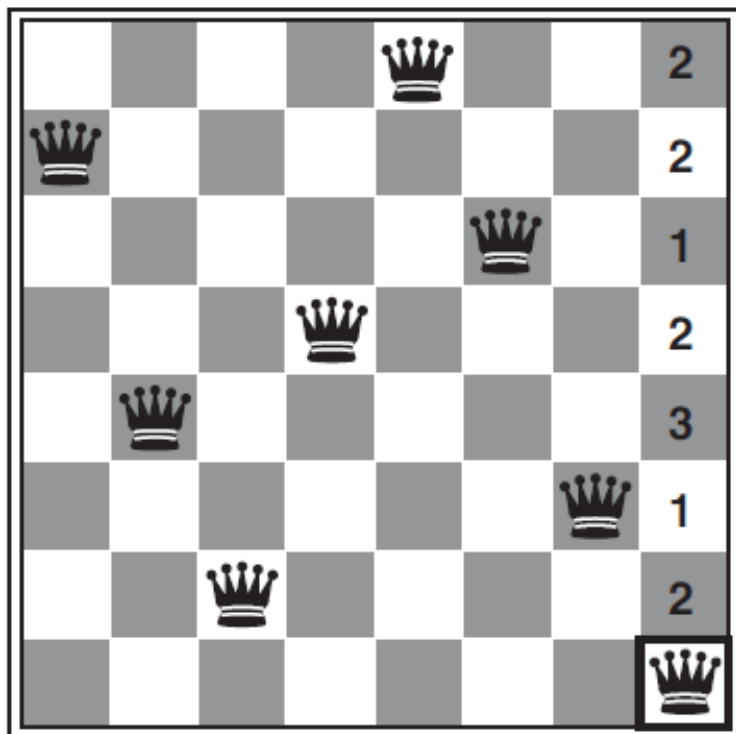
- 执行AC-3后，每一条弧都是一致的，或者某些变量的取值范围为空
- AC-3的复杂度：一共 c 条弧，每条要做 d^2 次检查，每条弧当弧首变量发生变化就检查，可以进入 d 次 $\rightarrow O(cd^3)$

- 连续空间的局部搜索 —— 梯度下降
- 约束满足问题不关心搜索的路径，只关心结果的状态
- 求解约束满足问题的核心：利用好结果信息，尽快排除不符合要求的结果 —— 约束传播
- 代表算法：回溯法（深度优先+变量排序+冲突停止）
- 维护弧一致：当一个变量的值域发生变化后，检查所有指向它的弧。要求弧尾值域中的任一取值，弧首值域中都有能跟它满足约束的取值，不然就从弧尾值域中删除这个取值

使用局部搜索求解

- 局部搜索使用完整的状态形式，即在初始状态时就给每个变量赋一个值，后继函数通常一次改变一个冲突变量的取值
- 最少冲突启发式：随机选择一个冲突变量，之后改变它的取值，选择与其他变量冲突最少的值
- 爬山法观察的是整个状态，最少冲突启发式观察单个冲突变量

使用局部搜索求解



- 逃出平台区：模拟退火、约束加权

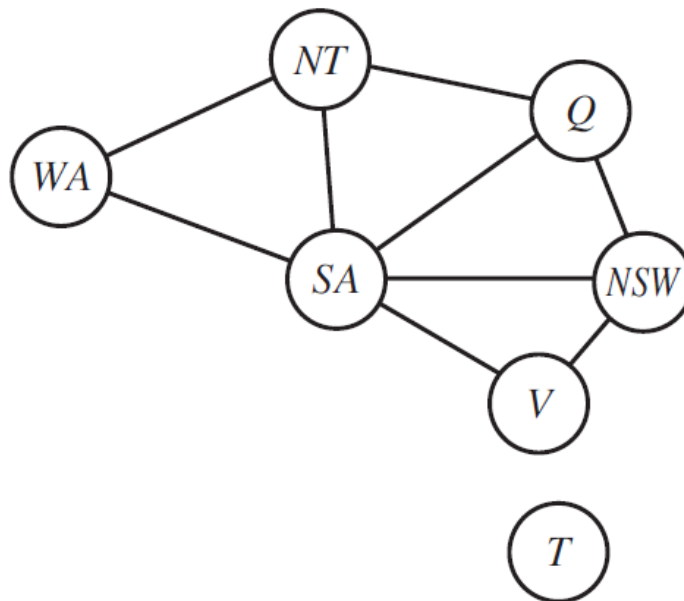
使用局部搜索求解

- 最少冲突启发式非常高效，它可以在50步左右解决百万皇后的问题
- 局部搜索的另一个优势是当问题改变时可以用于在线设置，这在调度问题中尤其重要：

比如一周的航班日程表可能涉及上千次航班和上万人次的分配，但是一个机场的恶劣天气可能就会打乱原来的日程安排。希望以最小的改动来修改日程，如果用一个使用新约束集的回溯搜索通常需要更多的时间，并且找到的解有可能要对当前日程进行很多改动

利用问题的结构信息求解

- 该问题可以分解为两个独立的子问题 CSP_i
- 如果 S_i 是子问题 CSP_i 的解，那么 $\cup_i S_i$ 是 $\cup_i CSP_i$ 的解



利用问题的结构信息求解

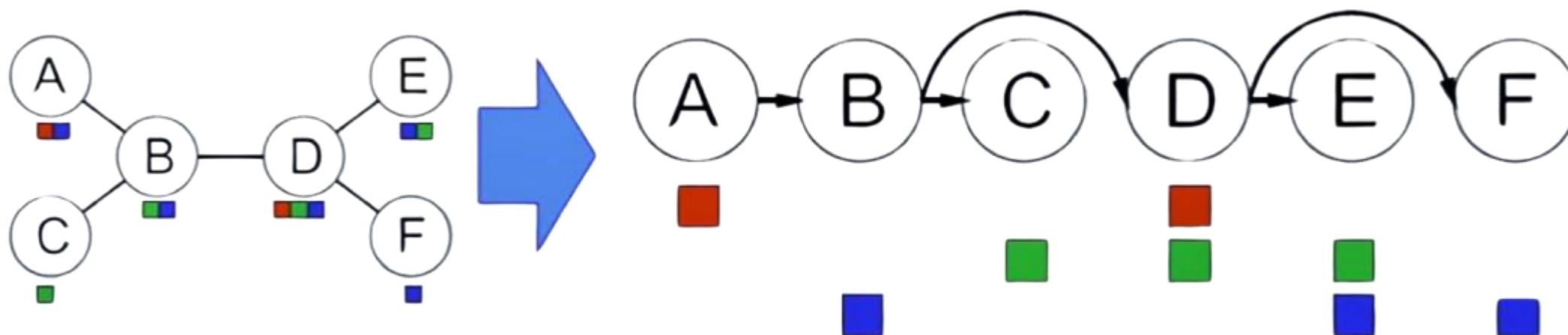
- 如果有 n 个变量， d 是值域， CSP_i 有 c 个变量
- $O(d^c (\frac{n}{c}))$ — 关于 n 是线性的 vs. $O(d^n)$ — 关于 n 是指数级的
- 例如：一个有80个变量，每个变量有2种取值的约束满足问题，如果能分解为4个子问题
- 假设每秒可以处理一千万个节点
- 原始问题的求解需要2的80次方/1kw=40亿年，分解后为2的20次方/1kw=0.1秒

利用问题的结构信息求解

- 独立的子问题很好，但是这种结构不常有
- 另一种好求解的结构：约束图为**树**——任意两个变量之间只有一条路径相连，即 n 个节点，有 $n-1$ 条边
- 树结构的约束满足问题可以在 $O(nd^2)$ 内求解
- 解决约束图是树结构的问题关键：**定向弧一致**，即考虑变量在给定的顺序下 $X_1, X_2, \dots, X_n$ ，每个 X_i 对 X_j ($j > i$) 是弧一致的

利用问题的结构信息求解

1. 选择一个节点作为根结点，按照父节点一定先于子节点的次序对节点进行排序—形成拓扑结构
2. 从孩子到父亲依次进行弧一致检查
 $D \rightarrow F$, $D \rightarrow E$, $B \rightarrow D$, $B \rightarrow C$, $A \rightarrow B$
3. 从根节点到叶子节点进行赋值: A B C D E F



- 约束满足问题不关心搜索的路径，只关心结果的状态，因此可以用局部搜索进行求解
- 约束满足问题的求解关键在于利用问题的结构信息，无论是约束传播，还是利用问题中特殊的结构，如连通分量和树